

Thursday 9:08 PM

why so strcpy(out->text, token); why not token_def->value[0]?"

Ah — if you're asking why I used `strcpy(out->text, token)` instead of directly doing something like `token_def->value[0] = token`, the key difference is that these are doing two completely different things.

```
token_def->value[0] = token
```

This puts **one character** into the first position:

```
token_def->value[0] = token
```

You would then need:

```
token_def->value[1] = token
```

to make it a valid C string:

```
return state == ST_START || state == ST_AFTER_WORD || state == ST_AFTER_AMP;
why do this?
```

Because after the `for` loop finishes, you need to check **whether the parser ended in a state where the input is allowed to end**.

Your grammar has:

```
LINE -> e | WORD ARG
ARG -> e | WORD ARG | OP_LT TGT | OP_ST TGT
      | OP_STGT TGT | OP_PIPE CMD | OP_SEMI CMD | OP_AMP BG
BG -> e | WORD ARG
```

So some states can legally be the **end of the input**, while others cannot.

These states are valid stopping points

```
ST_START
ST_AFTER_WORD
ST_AFTER_AMP
```

These states are **NOT** valid stopping points

```
ST_NEED_WORD_TGT
ST_NEED_WORD_CMD
```

For example:

```
ls >
```

After seeing `>`, you enter:

```
ST_NEED_WORD_TGT
```

But the grammar says:

```
OP_ST TGT
```

and:

```
yprya@fedora:~/mini-project1_2025114007/c-shell$ ./cshell
<user:yprya@fedora.dir~>hop /home/yprya/mini-project1_2025114007/c-shell/temp
<user:yprya@fedora.dir~>??
```

Okay — that confirms the **problem is not the path**. That absolute path definitely exists, so your `hop_one()` should do:

```
path_is_dir(arg) -- 1
target = arg
chdir(target) -- success
```

But your prompt stays at `>`.

The most likely problem is **your `main.c` is not actually using the changed process CWD to generate the prompt**, or the `hop()` call isn't being reached/executed as expected.

Let's isolate it.

1. Add ONE debug print in `hop_one()`

Immediately after:

```
if(chdir(target)==0){
    printf("hop: no such directory\n");
    return 1;
}
```

add:

```
printf("DEBUG: chdir succeeded, target = %s\n", target);
```

So:

```
if(chdir(target)!=0){
    printf("hop: no such directory\n");
    return 1;
}

printf("DEBUG: chdir succeeded, target = %s\n", target);
```

Compile:

```
Bash
gcc -Wall -Wextra -g main.c lexer.c parser.c hop.c -o cshell
```

Run:

Immediately after the `chdir()` add:

```
char debug_cwd[PATH_SIZE];

if(getcwd(debug_cwd, sizeof(debug_cwd)) != NULL)
{
    printf("DEBUG CWD = %s\n", debug_cwd);
}
```

So:

```
if(chdir(target)!=0){
    printf("hop: no such directory\n");
    return 1;
}

char debug_cwd[PATH_SIZE];

if(getcwd(debug_cwd, sizeof(debug_cwd)) != NULL)
{
    printf("DEBUG CWD = %s\n", debug_cwd);
}
```

Then you should get:

Your current `hop_one()` has:

```
void C
if(chdir(target)!=0){
    printf("hop: no such directory\n");
    return 1;
}
```

That is exactly where we want the actual directory change to happen.

So don't start changing the linked-list code.

First do the debug test above.

Send me the output of:

```
./cshell
hop /home/priya/mini-project1_2025114007/c-shell/temp
```

with the `DEBUG CWD` = ... line added.

```
tokens[count].type = OP_WORD;
tokens[count].text = text;

count++;
}
```

okay so the issue here is that the array has a fixed size. Can u modify teh code so it works for a vector or liex a linked list. ALSO please add a funtion to free all tokens at the end of it

Yes — for your current code, the cleanest change is to use the **vector version** rather than a linked list. This keeps the token storage contiguous while allowing it to grow dynamically.

I'll keep your lexing logic essentially the same and change only the fixed-size storage to a growable vector. I'll also include a proper function to free all token memory.

```
void C
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <lexer.h>

#define WORD_SIZE 32
#define TOK_SIZE 8

int is_special(char c)
{
    return c == "<" || c == ">" || c == ":" ||
           c == "!" || c == "&";
}
```

Yes — for your current code, the cleanest change is to use the **vector version** rather than a linked list. This keeps the token storage contiguous while allowing it to grow dynamically.

I'll keep your lexing logic essentially the same and change only the fixed-size storage to a growable vector. I'll also include a proper function to free all token memory.

```
void C
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <lexer.h>

#define WORD_SIZE 32
#define TOK_SIZE 8

int is_special(char c)
{
    return c == "<" || c == ">" || c == ":" ||
           c == "!" || c == "&";
}
```

```
void C
char *data;
int len;
int cap;
} word_t;

/*
 * Initialize word buffer.
 */
void word_init(word_t *w)
{
    w->cap = WORD_SIZE;
    w->len = 0;
    w->data = malloc(w->cap);
    if (w->data == NULL)
    {
        fprintf(stderr, "chell: out of memory\n");
        exit(1);
    }
    w->data[0] = '\0';
}
```

```
void C
free(list);
}
```

The important change

Your fixed version had:

```
void C
token_t tokens[TOK_SIZE];
int count = 0;
```

So after 8 tokens:

```
echo a b c d e f g
```

you would run out of space.

Now you have:

```
void C
typedef struct
{
    token_t *tokens;
    int count;
    int cap;
} token_vec_t;
```

Initially:

Today 6:38 PM

wait m confused. How exactly does the struct implementation work? can u trace through an example to explain this to me?

Absolutely. The confusing part is that there are **two different structs** here:

1. `token_vec_t` → stores the **array of tokens**
2. `word_t` → temporarily builds **one word**

Let's trace:

```
echo hello > out.txt
```

1. What is `token_vec_t`?

You defined:

```
void C
typedef struct
{
    token_t *tokens;
    int count;
    int cap;
}
```

```

|
|-----|
| [0]    |
| type = OP_WORD |
| text --> "echo" |
|-----|
| [1] EMPTY |
|-----|
| [2] EMPTY |
| ...      |
|-----|
| [7] EMPTY |
|-----|
```

Finally:

```
void C
v->count++;
```

So:

```
count = 1
cap = 8
```

Meaning:

"I have 1 token stored, and I have room for 8."

`lex_word()` builds:

```
"echo"
```

and returns a dynamically allocated string.

Then:

```
void C
tv_push(&vec, OP_WORD, text);
```

4. What does `tv_push()` do?

The function starts:

```
void C
void tv_push(token_vec_t *v, token_category type, char *text)
```

We passed:

```
void C
```

8. So where does the "vector" part actually happen?

Suppose the command is much longer and we reach:

```
count = 8
cap = 8
```

The array is full:

```
[0] used
[1] used
[2] used
[3] used
[4] used
[5] used
[6] used
[7] used
```

Then we try another:

```
on C
tv_push(&vec, ...);
```

This condition becomes true:

```
on C
if (v->count == v->cap)
```

Yes. Looking at your **current** `lexer.c` and `main.c`, the main structural change is that your lexer no longer produces a `token_def *` array. It produces a `token_list_t *`, whose structure is:

- `list->tokens` → array of `token_t`
- `list->count` → number of tokens

Your lexer does this in `lex_line()` and returns `token_list_t *`. [lexer](#)
It then stores the array/count into the list here. [lexer](#)

So we don't need to rewrite your parser logic. We just change how it accesses the tokens.

Minimal-edited `parser.c`:

```
on C
#include "parser.h"

// LINE -> ε
// | WORD ARG

// ARG -> ε
// | WORD ARG
// | OP_LT TST
// | OP_GT TST
// | OP_STGT TST
// | OP_PIPE CHD
// | OP_SEMT CHD
// | OP_AMP BG

// CHD -> WORD ARG
// TGT -> WORD ARG
```

```
if(tokens[i].type == OP_WORD)
{
    curr_state = STATE_AFTER_WORD;
}
else return 0;
break;
}

if (curr_state == STATE_S || curr_state ==
STATE_OPERATOR_AND_OR || curr_state == STATE_AFTER_WORD)
{
    return 1;
}
return 0;
}
okay so this is my parser.c, but i had to edit the project structure .
Can u make MINIMAL edits to it so that it works with my current
project structure? I've uploaded my lexer.c and main.c for reference
```

Yes. Looking at your **current** `lexer.c` and `main.c`, the main structural change is that your lexer no longer produces a `token_def *` array. It produces a `token_list_t *`, whose structure is:

- `list->tokens` → array of `token_t`
- `list->count` → number of tokens

Your lexer does this in `lex_line()` and returns `token_list_t *`. [lexer](#)
It then stores the array/count into the list here. [lexer](#)

So we don't need to rewrite your parser logic. We just change how it accesses the tokens.

```
on C
token_category token_type_parser(const parser_t *p)
{
    return p->tokens[p->pos].type;
}

int ARG(parser_t *p);
int CHD(parser_t *p);
int TGT(parser_t *p);
int BG(parser_t *p);

// LINE -> ε
// | WORD ARG
// ARG -> ε
// | WORD ARG
// | OP_LT TST
// | OP_GT TST
// | OP_STGT TST
// | OP_PIPE CHD
// | OP_SEMT CHD
// | OP_AMP BG
// CHD -> WORD ARG
// TGT -> WORD ARG
// BG -> ε
// | WORD ARG

int CHD(parser_t *p) {
    if (at_end(p) || token_type_parser(p) != OP_WORD)
    {
        return 0; //if
    }
    p->pos++;
    return ARG(p);
}
```

```
on C
return CHD(p);
case OP_SEMT:
    p->pos++;
    return CHD(p);
case OP_AMP:
    p->pos++;
    return BG(p);
default:
    return 0;
}

/* LINE -> ε | WORD ARG */
int LINE(parser_t *p) {
    if (at_end(p)) {
        return 1;
    }
    if (token_type_parser(p) != OP_WORD) {
        return 0;
    }
    p->pos++;
    return ARG(p);
}

int parse_tokens(const token_list_t *list) {
    parser_t p = { .toks = list->tokens, .n = list->count, .pos = 0 };
    return LINE(&p);
}
```

wait wat? can u trace through the code with an examples?

Yes 🤖. The easiest way to understand this parser is to forget the C code for a second and trace it as the grammar.

Let's use:

```
echo hello > output.txt
```

Your lexer will produce something conceptually like:

Token	Type
echo	OP_WORD
hello	OP_WORD
>	OP_ST
output.txt	OP_WORD

So:

```
toks[0] = OP_WORD ("echo")
toks[1] = OP_WORD ("hello")
toks[2] = OP_ST (">")
toks[3] = OP_WORD ("output.txt")
```

1. LINE(&p)

Your grammar says:

```
LINE -> ε
| WORD ARG
```

We aren't at the end:

```
on C
if (at_end(p))
```

means:

```
on C
return p->pos >= p->n;
```

Currently:

```
0 >= 4 - false
```

So we continue.

Then:

```
on C
if (token_type_parser(p) != OP_WORD)
```

Now() again

Now:

```
pos = 2
token = OP_ST
```

Again:

```
on C
if (at_end(p))
```

gives:

```
2 >= 4 - false
```

Then:

```
on C
switch(token_type_parser(p))
```

gives:

```
OP_ST
```

So we enter:

```
on C
```

The entire trace visually

This is probably the most useful way to think about it:

```
parse_tokens()
|
v
LINE
|
| sees WORD "echo"
| pos = 0 - 1
|
v
ARG
|
| sees WORD "hello"
| pos = 1 - 2
|
v
ARG
|
| sees OP_ST "~"
| pos = 2 - 3
|
v
TGT
|
| sees WORD "output.txt"
| pos = 3 - 4
|
v
ARG
|
| end of tokens
|
v
1
```

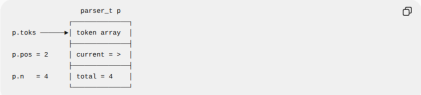
One REALLY important thing about your parser_t

This part:

```
typedef struct {
    const token_t *toks;
    int pos;
    int n;
} parser_t;
```

is basically your parser's **notebook**.

For our example:



The most important variable is:

```
typedef struct {
    const token_t *toks;
    int pos;
    int n;
} parser_t;
```

Today 6:45 PM

```
wait s wat does int CMD(parser_t *p) {
    if (at_end(p) || token_type_parser(p) != OP_WORD)
    {
        return 0;
    }
    p->pos++;
    return ARG(p);
} do exactly?
```

Exactly. `CMD()` is basically the parser's function for checking this grammar rule:

```
CMD -> WORD ARG
```

Your code:

```
int CMD(parser_t *p) {
    if (at_end(p) || token_type_parser(p) != OP_WORD)
    {
        return 0;
    }
    p->pos++;
    return ARG(p);
}
```

```
typedef struct {
    const token_t *toks;
    int pos;
    int n;
} parser_t;
```

For example, suppose the input is:

```
echo hello | wc
```

The tokens are:

index	0	1	2	3
token	echo	hello		wc
type	WORD	WORD	PIPE	WORD

If we're trying to parse the `wc` part:

```
echo hello | wc
1
pos = 3
```

So `p` tells `CMD()`:

"Here's the entire token array, and I'm currently looking at token 3."

2. This checks whether a command actually starts with a WORD

```
int CMD(parser_t *p) {
    if (at_end(p) || token_type_parser(p) != OP_WORD)
    {
        return 0;
    }
    p->pos++;
    return ARG(p);
}
```

Remember your grammar:

```
CMD -> WORD ARG
```

So `CMD()` must see a `WORD` first.

For:

```
echo hello | wc
```

when `CMD()` is called after `|`:

```
pos = 3
token = wc
type = OP_WORD
```

Therefore:

```
int CMD(parser_t *p) {
    if (at_end(p) || token_type_parser(p) != OP_WORD)
    {
        return 0;
    }
    p->pos++;
    return ARG(p);
}
```

4. Then return ARG(p)

This is the important part.

The grammar says:

```
CMD -> WORD ARG
```

We've already consumed the `WORD`.

So now we need to parse the `ARG` part.

That's why:

```
int ARG(parser_t *p) {
    return ARG(p);
}
```

means:

"I've successfully parsed the `WORD`. Now you parse whatever comes after it according to the `ARG` rule."

Let's trace `echo hello | wc -l`

Tokens:

index	0	1	2	3	4
token	echo	hello		wc	-l
type	WORD	WORD	PIPE	WORD	WORD

```
{
    argv[j]=array_strings[j].value;
    j++;
}
}

hop(argc, argv, getenv("HOME"));
free(argv);
}

free(input_line);
free(array_strings);
}

int main()
{
    char home_dir(PATH_MAX);
    getcwd(home_dir, sizeof(home_dir));

    input_loop(home_dir);
    return 0;
}
okay so this is my main.c code for my PREVIOUS lexerc and
parser.c (I have uploaded them before in the chat). DO NOT
CHANGE THE WORKING OF MAIN.C and please edit the code so
that it works for my PRESENT architecture. DO NOT CHANGE MY
lexer or parser.
```

```
return 0;
} okay so this is my main.c code for my PREVIOUS lexerc and
parser.c (I have uploaded them before in the chat). DO NOT
CHANGE THE WORKING OF MAIN.C and please edit the code so
that it works for my PRESENT architecture. DO NOT CHANGE MY
lexer or parser.
```

Yes. We'll **only** modify `main.c`.

Your current architecture is:

```
main.c
|
lex_line()
|
token_list_t
|
tokens
|
count
|
parse_tokens()
```

Your current lexer already returns `token_list_t *` from `lex_line()` and has `free_token_list()` for cleanup.

So the old `read_line()`, `split_line()`, `token_def_t`, `do_stuff()`, and `op_eof` array handling need to go **only** because they belong to the old architecture.

Here is your `main.c` with the **minimum structural changes**, while keeping the behavior of your main loop, prompt, hop, syntax-error handling, etc. th

```
while (1)
{
    char_read=getchar();

    if (char_read==EOF||char_read=='\n')
    {
        in_string[curr]='\0';
        return in_string;
    }
    else
    {
        in_string[curr]=char_read;
    }
    curr++;

    if (curr==buffer_size)
    {
        buffer_size=const_buffer_size;
        in_string=realloc(in_string,sizeof(char)*buffer_size);

        if (!in_string)
        {
            fprintf(stderr, "memory allocation error");
            exit(1);
        }
    }
}

/* input loop */
void input_loop(char* home_dir)
{

```

```
{
    char home_dir[MAX_PATH];
    getcwd(home_dir, sizeof(home_dir));
    input_loop(home_dir);
    return 0;
}
```

What I changed — and only what was necessary

Your old code had:

```
on C
token_buf = array_strings;
```

Now it has:

```
on C
token_list_t* token_list;
```

because your current lexer returns:

```
on C
token_list_t *lex_line(const char *line, int *error)
```

and the list contains:

```
on C
int error=0;
token_list=lex_line(input_line,error);
```

The old:

```
on C
if (!parse(array_strings))
```

becomes:

```
on C
if (!parse_tokens(token_list))
```

because your **present** parser has:

```
on C
int parse_tokens(const token_list_t *list)
```

And this old:

```
on C
array_string_t* run;
```

stage, and the rest of the pipeline still runs.

9. File redirection and pipes must work together (e.g. `command1 < input.txt | command2 > output.txt`).

Example

```
<goatam@liit:--> printf "banana\apple" | sort
apple
banana
<goatam@liit:--> badcmd | sort
cshell: command not found (badcmd)
<goatam@liit:--> cat < in.txt | sort > out.txt
```

okay so like this is what i need to implement. Can u explain STEP BY STEP how i need to do it. Like wat are the pinters the file shld do. Like i know one thing is checking if its actually executable or not. THEN?

Yes. Think of C1-C4 as **one** execution pipeline that gets progressively more complicated.

You already have the lexer/parser. So the next part is basically:

```
tokens
|
understand command + redirections + pipes
|
find executable
|
```

```
i
execv()
```

And yes — checking whether the file is executable is **only** the first part.

1. First understand what `execvte.c` is supposed to receive

Suppose the user types:

```
on Bash
cat < input.txt | sort > output.txt
```

Your lexer/parser has already converted this into something conceptually like:

```
WORD cat
OP_LT
WORD input.txt
OP_PIPE
WORD sort
OP_GT
WORD output.txt
EOF
```

The parser's job is only:

```
| "Is this syntactically valid?"
```

The executor's job is:

3. C1: Find the executable

This should happen **before** `fork()` if you're doing a normal single command, and before executing each child in a pipeline.

Suppose:

```
on Bash
ls
```

The command name is:

```
on C
argv[0] = "ls";
```

You need to determine:

```
What actual file should execv() run?
```

There are three cases.

Case A — command contains `/`

Example:

you eventually do:

```
on C
execv("./hello", argv);
```

4. Case B — no `/`

Example:

```
on Bash
hello
```

First check the **current** directory.

If your current directory contains:

```
./hello
```

and it is executable:

```
on C
access("./hello", X_OK)
```

then use:

```
Hello
|
Path directly
```

6. Searching PATH

Get PATH:

```
on C
char *path = getenv("PATH");
```

It looks something like:

```
/usr/local/bin:/usr/bin:/bin
```

You need to split it at `:`.

Conceptually:

```
PATH
|
/usr/local/bin
/usr/bin
/bin
```

Then for each directory construct:

This is important.

Suppose:

```
bash
echo hello > output.txt
```

You find:

```
usr/bin/echo
```

But don't fork/exec yet.

First deal with the redirections.

Why?

Because your assignment says:

If a redirection file cannot be opened, don't execute the command.

So:

```
find executable
|
| validate/open redirection files
|
| ONLY THEN fork()
```

13. C3: Output redirection

Suppose:

```
bash
echo hi > output.txt
```

Open:

```
C
open("output.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644);
```

Suppose it gives:

```
fd = 3
```

Then child:

```
C
dup(3, STDOUT_FILENO);
```

Now:

```
stdout
```

Child 2:

```
needs READ
doesn't need WRITE
```

SO:

```
C
close(pipefd[1]);
```

Parent:

```
doesn't need either
```

SO:

```
C
close(pipefd[0]);
close(pipefd[1]);
```

Otherwise programs like `sort` can sit waiting because some process still has the write end open.

20. Pipeline with 3 commands

For:

And the `execute.c` architecture I'd use

Ultimately, you want something like:

```
execute_pipeline()
|
| split tokens into commands
|
|
| command 1
|   |-- argv
|   |-- input redirections
|   |-- output redirections
|
| command 2
|   |-- argv
|   |-- input redirections
|   |-- output redirections
|
| ...
|
| find_executable()
|
| validate redirections
|
| create pipes
|
| fork each command
|
| child:
|   |-- setup stdin
|   |-- setup stdout
|   |-- close unused fds
|   |-- execv()
```

```
YES
|
|
| Does stat() work?
|
| NO --> return 0
|
| YES
|
| Is it a directory?
|
| YES --> return 0
|
| NO
|
|
| return 1
```

So:

Path	Result
Executable file	1
Non-executable file	0
Directory	0
Doesn't exist	0
Inaccessible path	0

okay so i need function that takes my file and then runs it to get these flags. A DIR is not an executable pls remember that. How do i implement that. Also can u explain the process pls?

Yes — and the important requirement is:

A directory must ALWAYS return 0, even if the directory has execute (x) permission.

If by "takes my file and then runs it to get these flags" you mean takes a filepath and determines whether it is an executable file, then you can implement exactly that with `access()` + `stat()`.

Function

```
C
#include <unistd.h>
#include <sys/stat.h>

int execute_is_executable(const char *filepath)
{
    struct stat st;

    /* First: does the path have execute permission? */
    if (access(filepath, X_OK) != 0)
        return 0;

    /* Second: get information about the path */
    if (stat(filepath, &st) != 0)
        return 0;
}
```

How the process works

Suppose you call:

```
C
int flag = execute_is_executable("./myprogram");
```

The function goes through three checks:

```
"/myprogram"
|
|
| Does it have X permission?
|
| NO --> return 0
|
| YES
|
| Can stat() read it?
|
| NO --> return 0
|
| YES
|
| Is it a directory?
|
```

Based on can u make a .h file for linking it to the rest of my projects?

Yes. Based on your `locate.h` structure, if this function is going into something like `execute.c`, you can make `execute.h`:

```
C
#ifndef EXECUTE_H
#define EXECUTE_H

#include <sys/stat.h>
#include <unistd.h>

int execute_is_executable(const char *filepath);

#endif
```

Then your files can be connected like this:

```
main.c
|
|-----> locate.h / locate.c
|
|-----> execute.h / execute.c
```


okay so here is the struct you shd use. typedef struct {

char path[PATH_MAX]; //path of directory

int frequency; //number of hits

long last_access; //time of hop

frequency_entry_t; NOW, can u write code so that 1. It saves in the files in everything from the db_array till the db_count position . So basically, the file has the format of db.frequency, db.last_access and db.path/ Also, BASED on this file structure, can u write a load history function that opens the file and when ur db_count is less than max entries it and reads the DIR NAME ONLY, Not the freq and last_access and then copies all the dir name to the array that says db and then increases the freq as well as the last access time

Yes. With your struct:

```
typedef struct {
    char path[PATH_MAX]; // path of directory
    int frequency; // number of hits
    long last_access; // time of hop
} frequency_entry_t;
```

I understand what you want as:

save_history()

okay so here's the thing, I have a way to save stuff into the file and to get it from the file, so like, now I have the logic of: int compare_entries(const frequency_entry_t *a, const frequency_entry_t *b)

```
{
    if (b->frequency != a->frequency)
    {
        return b->frequency - a->frequency;
    }

    if (b->last_access > a->last_access)
    {
        return 1;
    }

    if (b->last_access < a->last_access)
    {
        return -1;
    }
}
```

return 0;

to compare diff entries. NOW how do I like update stuff and call the save history function. Like can u give me a function to do that?

Yes. Since you already have:

- db[] containing your frequency_entry_t entries
- db_count
- compare_entries()
- save_history()

you want one function that says:

"This directory was accessed. Update its frequency/time if it already exists; otherwise add it. Then save the updated database to the file."

That's exactly what your update_frequency() function should do.

```
void update_frequency(const char *abs_path)
{
    /* Check if this directory already exists */
    for (int i = 0; i < db_count; i++)
    {
        if (strcmp(db[i].path, abs_path) == 0)
        {
            /* Existing entry */
            db[i].frequency++;
            db[i].last_access = (long)time(NULL);
            save_history();
            return;
        }
    }
}
```

The loader does:

```
5 -- ignored
1755943200 -- ignored
/home/user/Documents -- copied into db[i].path
```

Then:

```
db[0].frequency = 1;
db[0].last_access = time(NULL);
```

So after loading:

```
db[0].path = "/home/user/Documents"
db[0].frequency = 1
db[0].last_access = current time
```

The old 5 and 1755943200 are not used.

BUT — if by "increase the frequency" you mean this:

Suppose the file says:

```
5 1755943200 /home/user/Documents
```

and you want loading to produce:

```
frequency = 6
last_access = current time
```

```
void load_history(void) {
    FILE *fp = fopen(history_path, "r");
    if (fp == NULL) {
        return;
    }
    db_count = 0;
    char line[PATH_MAX + 128];
    while (db_count < MAX_ENTRIES && fgets(line, sizeof(line), fp) != NULL) {
        int freq;
        long last_acc;
        if (sscanf(line, "%ld %ld", &freq, &last_acc) == 2) {
            char *p = line;
            while (*p && (*p == ' ' || *p == '\t')) p++;
            while (*p && (*p == '0' || *p <= '9')) p++;
            while (*p && (*p == ' ' || *p == '\t')) p++;
            while (*p && ((*p == '0' || *p <= '9') || (*p == '-' || *p == '+')) p++;
            while (*p && (*p == ' ' || *p == '\t')) p++;

            size_t plen = strlen(p);
            while (plen > 0 && (p[plen - 1] == '\n' || p[plen - 1] == '\r')) {
                p[plen - 1] = '\0';
                plen--;
            }

            if (plen > 0) {
                db[db_count].frequency = freq;
                db[db_count].last_access = last_acc;
                strcpy(db[db_count].path, p, PATH_MAX - 1);
                db[db_count].path[PATH_MAX - 1] = '\0';
                db_count++;
            }
        }
    }
}
```

```
5 1755943200 /home/user/Documents
```

load_history()

When reading the file, you don't want to restore the old frequency and last_access values.

Instead, you want to:

- read the directory/path
- put it into db
- increase its frequency
- set/update its last_access to the current time

There is one important issue, though: if your file is:

```
5 1755943200 /home/user/Documents
```

you cannot simply use fgets() and treat the whole line as the directory, because the frequency and timestamp are also there.

So we need to skip the first two fields and extract only the path.

Here's the clean version:

```
void save_history(void)
{
    FILE *fp = fopen(history_path, "w");
    if (fp == NULL) {
        return;
    }
}
```

```

/* Database is full: replace the lowest-ranked entry */
int min_idx = 0;

for (int i = 1; i < db_count; i++)
{
    if (compare_entries(&db[i], &db[min_idx]) > 0)
    {
        min_idx = i;
    }
}

strcpy(db[min_idx].path, abs_path, PATH_MAX - 1);
db[min_idx].path[PATH_MAX - 1] = '\0';

db[min_idx].frequency = 1;
db[min_idx].last_access = (long)time(NULL);
}

/* Save the updated database to the file */
save_history();
}

```

How it works

Suppose your database currently contains:

```

db[0] - /home/user
frequency = 5
last_access = 1000

db[1] - /home/user/Documents
frequency = 3
last_access = 2000

```

How it works

Suppose your database currently contains:

```

db[0] - /home/user
frequency = 5
last_access = 1000

db[1] - /home/user/Documents
frequency = 3
last_access = 2000

```

You call:

```

update_frequency("/home/user");

```

The function finds /home/user:

```

strcmp(db[0].path, abs_path) == 0

```

Then:

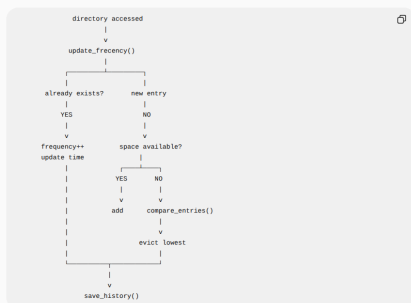
```

db[0].frequency++;

```

Your `compare_entries()` tells you which entry has the lower frequency ranking.

So the overall architecture is:



So you don't need to call `save_history()` separately every time you update an entry. Put it inside `update_frequency()` at the end, and whenever your program records a directory access, just call:

okay so i added the `save_history`, `update_history` and `update_frequency` functions. Based on my project requirements, what more shld i do?

Yes. Based on the requirements you showed, `save_history()` + `load_history()` + `update_frequency()` are **only** the frequency/database part. You still need to implement the actual `hop` behavior around them.

You already have the important building blocks, so I'd divide what's left into 4 main functions:

1. `compare_entries()` — you already have this ✓

You have:

```

int compare_entries(const frequency_entry_t *a, const frequency_entry_t *b)
{
    if (b->frequency != a->frequency)
    {
        return b->frequency - a->frequency;
    }
    if (b->last_access > a->last_access)
    {
        return -1;
    }
    return 0;
}

```

```

return -1;
}
return 0;
}

```

This determines which directory has the **higher** frequency ranking.

Your ranking is:

- Higher `frequency` — better
- If frequency is equal — more recent `last_access` — better

So this satisfies:

"Standing depends on both frequency and recency."

2. `frequency_lookup()` — you still need this ✗

This is needed for requirement 6:

If "`name`" does not resolve directly, fall back to a frequency lookup.

And requirement 3 under Frequency:

Find the highest-ranked directory whose path contains "`name`" as a substring.

For example, suppose your database contains:

```

/home/user/Documents/projects

```

or C

```

strcmp(db[1].path, name)

```

checks whether `name` occurs inside the path.

Step C — make sure the directory still exists

Use:

```

or C
stat()

```

and:

```

or C
S_ISDIR(st)

```

This is required because the specification explicitly says:

If the top match no longer exists on disk, skip it and try the next best.

So yes, this function is **definitely** something you still need to implement.

3. `do_chdir()` — you still need this ✗

This is an important helper because every successful directory change needs to update frequency.

5. The main `hop()` function — this is the biggest remaining part ✗

This is where you implement the **actual** requirements 1–7.

Your `hop()` needs to process arguments **sequentially**.

For example:

```

hop go team ...

```

means:

```

process "go"
{
    process "team"
    {
        process "...", ...
    }
}
process "...", ...

```

not just process one argument.

Your `hop()` should handle these cases

Case 1: No arguments

And your `do_chdir()` should update `frequency` before the change.

6. Direct path lookup — still needed ✗

Requirement 5 says:

If "`name`" resolves to a valid relative or absolute path from the CWD, change to it directly.

This is VERY important.

For example:

```

hop Documents

```

```

if
Documents

```

exists, you should **directly** `chdir()` there.

You don't do frequency lookup first.

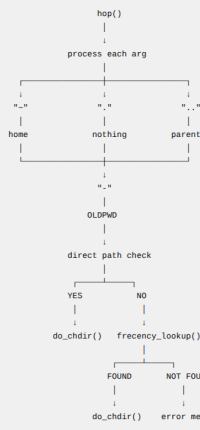
So the order must be:

```

if
Documents

```

You still need to connect that database to the actual `hop` command:



```
frequency_lookup() -> do_chdir() -> hop_init() -> hop()
```

Once those four are connected to your existing functions, you have essentially implemented the whole `hop` requirement.

🔗 📄 🔄 ⋮

Today 12:34 AM

```
for (int i = 0; i < db_count; i++)
{
    int idx = order[i];

    if (strcmp(db[idx].path, name) != NULL) {
        struct stat st;

        if (stat(db[idx].path, &st) == 0 && S_ISDIR(st.st_mode)) {
            return db[idx].path;
        }
    }
}
```

what does this code do?

Meaning:

```
highest ranked - db[1]
second         - db[2]
lowest        - db[9]
```

So:

```
int idx = order[i];
```

gets the actual `db` index.

For `i = 0`:

```
idx = order[0]
= 1
```

Therefore:

```
db[idx]
```

means:

```
db[1]
```

1. Add `hop_init()`

Put this after `do_chdir()` or before it — function order doesn't matter if you put prototypes in `hop.h`.

```
void hop_init(const char *home_dir)
{
    /* Store the shell's home directory */
    strcpy(hop_shell_home, home_dir, PATH_MAX - 1);
    hop_shell_home[path_max - 1] = '\0';

    /*
     * Put the history file in the same directory
     * as the shell executable.
     */
    char exe_path[PATH_MAX];

    ssize_t len = readlink("/proc/self/exe",
                          exe_path,
                          sizeof(exe_path) - 1);

    if (len > 0)
    {
        exe_path[len] = '\0';

        char *last_slash = strrchr(exe_path, '/');

        if (last_slash != NULL)
        {
            /*
             * Remove the executable name.
            */
        }
    }
}
```

```
for (int i = 0; i < db_count; i++)
```

This loops through every entry in your database.

If:

```
db_count = 4;
```

then:

```
i = 0
i = 1
i = 2
i = 3
```

Remember: `db` contains your frequency entries.

1. Get the actual database index

```
int idx = order[i];
```

This is important because `order` contains the indexes of `db` in sorted order.

For example, your actual database might be:

```
{
    update_frequency();
}

if (prev[0] != '\0')
{
    setenv("OLDPWD", prev, 1); //std OLD PWD FOR hop...
}
return 1;
} //these are all jst functions. How do I implement them together so
that the hop actually works. Can u give me code for that AND
EXPLAIN it pls.
```

Yes, you have already written almost all the individual pieces. What you're missing is the main `hop()` function that connects them together, plus an initialization function that sets the home directory/history path and calls `load_history()`.

There is also one naming issue in your code:

```
char hop_shell_path[PATH_MAX];
```

but the earlier code used `hop_shell_home`. Since `hop` needs the shell's home directory, I would rename this to:

```
char hop_shell_home[PATH_MAX];
```

```
find_hop_history
load_history()
db[] now contains previous hops
```

So `hop_init()` should be called once when your shell starts.

2. Now implement the actual `hop()`

This is the important part.

```
void hop(const token_list_t *list)
{
    /* No arguments
     * hop
     * behaves like
     * hop -
     */
    if (list->count == 1)
    {
        if (!strcmp(hop_shell_home))
        {
            printf("hop: no such directory\n");
        }
    }
}
```

```
        continue;
    }

    const char *arg = list_tokens[i].text;

    /*
     * .....
     * 1. "-"
     * .....
     */
    *hop =
    /* Go to shell's home directory.
     */
    if (strcmp(arg, "-") == 0)
    {
        if (!do_chdir(hop_shell_home))
        {
            printf("hop: No such directory\n");
        }
        continue;
    }

    /*
     * .....
     * 2. "."
     * .....
     */
    *hop =
```

```
.....
*
* = hop =
* Go to the previous working directory.
*/
if (strcmp(arg, "-") == 0)
{
    char *old = getenv("OLDPWD");

    if (old != NULL)
    {
        if (!do_chdir(old))
        {
            printf("hop: No such directory\n");
        }
    }
    continue;
}

/*
* .....
* 3. Direct path
* .....
*
* First try the argument as an ordinary
* relative or absolute directory path.
*
* Examples:
*
* hop Documents
* hop ./Documents
```

```
on C
{
    ...
}
```

is a **standalone block**.
It doesn't mean "if".
It's mainly being used here to create a **local scope**.
For example:

```
on C
{
    int x = 10;
}
/* x doesn't exist here */
```

x only exists inside that **{ }**.
In your code, this:

```
on C
struct stat st;
```

```
on C
hop_init()
hop()
```

They aren't independent. They form a chain.

When the shell starts
Your main shell should call:

```
on C
hop_init(home_dir);
```

For example, if your shell already has:

```
on C
char home_dir[MAX_PATH];
```

then:

```
on C
hop_init(home_dir);
```

happens **once** during initialization.
That causes:

```
on C
hop_init()
```

2. What does `stat()` do?

This:

```
on C
struct stat st;
```

creates a structure that will contain information about a filesystem object.
Then:

```
on C
stat(arg, &st)
```

asks Linux:
"Tell me about the file/directory represented by `arg`."
For example, if:

```
arg = "projects"
```

then:

```
on C
```

```
/*last_stack = 0*/;
strcpy(history_path, exe_path, PATH_MAX - 15);
history_path[PATH_MAX - 15] = '\0';
strcat(history_path, "/"hop_history");
} else {
    snprintf(history_path, sizeof(history_path),
"%s/"hop_history", home_dir);
}
} else {
    snprintf(history_path, sizeof(history_path), "%s/"hop_history",
home_dir);
}

load_history();
} wat wat does this code do line by line can u explain it to me in
easy terms
```

Absolutely. This function is basically the **setup function** for `hop`.
Its job is:

1. Remember what the shell's home directory is.
2. Figure out where your shell executable is.
3. Build the path to `./hop_history`.
4. Load the old frequency data from that file.

Let's go **line by line** in **very simple terms**.

That's basically the entire purpose of `-t`.

3. `-t` means recursive

Without `-t`:

```
reveal
```

might show:

```
Documents
Downloads
file.txt
```

But it **doesn't enter** `Documents` or `Downloads`.
With:

```
reveal -t
```

you need:

```
Documents
assignment1
assignment2
Downloads
```

Downloads

8. You MUST sort the entries

Your requirement says:

Entries are always listed in lexicographic order, sorted by ASCII value.

Don't rely on the order returned by `readdir()`.
You need to collect the names first.
For example, you could create:

```
on C
char *names[MAX_ENTRIES];
int count = 0;
```

and while reading:

```
on C
names[count] = strdup(entry->d_name);
count++;
```

Then sort them with `strcmp()`.
For example:

For example:

```
entry->d_name
```

could be:

```
(".",
"..",
"documents",
"downloads",
"hello.txt")
```

7. Implementing `-a`

Suppose:

```
on C
int show_all = 0;
```

means `-a` wasn't provided.

Then:

```
on C
if (!(show_all && entry->d_name[0] == '.'))
{
    continue;
}
```

```
└─ read directory
└─ sort entries
└─ print entries
└─ recursively call itself if -t
```

This will make your code MUCH easier to understand.

5. Opening a directory

Inside the helper, you'll use:

```
on C
DIR *dir = opendir(path);
```

You'll need:

```
on C
#include <dirent.h>
```

For example:

```
on C
DIR *dir = opendir("/home/user");
```

If it succeeds, you can read its entries.

If it fails:

Yes — that is actually the cleaner design.

Your `hop` already has the logic for resolving:

- no argument / `.`
- `..`
- direct relative path
- direct absolute path
- frequency fallback

For `reveal`, you want the exact same resolution logic EXCEPT frequency.

So:

```
hop:
argument
├─ . / - / .. / -
├─ direct path
└─ frequency
```

while `reveal` should be:

Today 1:04 AM

okay so u have my hop code for resolving the idirectory no. Can u remove the frequency part from it and use it for reveal too. Also will that logic even be correct? Or will it not work??

Yes — that is actually the cleaner design.

Your `hop` already has the logic for resolving:

- no argument / `.`
- `..`
- `..`
- direct relative path
- direct absolute path
- frequency fallback

For `reveal`, you want the exact same resolution logic EXCEPT frequency.

So:

```
hop:
argument
├─ . / - / .. / -
├─ direct path
└─ frequency
```

That avoids having two copies of the same logic.

What the shared function should do

Based on the `hop` logic you've shown me, it should be:

```
on C
int resolve_directory(const char *arg,
                    const char *home_dir,
                    char *resolved,
                    size_t size)
{
    char cwd[PATH_MAX];

    /* Get current working directory */
    if (getcwd(cwd, sizeof(cwd)) == NULL)
        return 0;

    /* No argument or "." - current directory */
    if (arg == NULL || strcmp(arg, ".") == 0)
    {
        strcpy(resolved, cwd, size - 1);
        resolved[size - 1] = '\0';
        return 1;
    }

    /* .. - home directory */
```

```
/*
 * This part depends on how YOUR hop stores the
 * previous directory.
 */
/* If hop uses OLDPWD:
 */
char *old = getenv("OLDPWD");

if (old == NULL)
    return 0;

strcpy(resolved, old, size - 1);
resolved[size - 1] = '\0';

return 1;

/* Direct relative or absolute path */
{
    char *rp = realpath(arg, NULL);

    if (rp == NULL)
        return 0;

    struct stat st;

    if (stat(rp, &st) != 0 || !S_ISDIR(st.st_mode))
    {
        free(rp);
        return 0;
    }
}
```

If that's still how your actual `hop.c` works, do NOT use `getenv("OLDPWD")` in the shared function.

Instead, the resolution function needs access to the same `prev_cwd`.

That's why I wouldn't blindly replace your existing code yet.

The correct architecture would be:

```
hop.c
├─ prev_cwd
├─ resolve_directory()
├─ ..
├─ ..
├─ direct path
├─ hop()
├─ resolve_directory()
└─ chdir()
```

and:

```
reveal.c
├─ reveal()
```

```
return 0;
}
printf("resolved: %s", rp);
free(rp);
return 1;
}
// can u explain this code line by line
```

Yep. Let's go line by line, and I'll also explain why each part is there. This function's job is:

Take whatever the user typed as a target (`.`, `..`, `..`, `..`, or a path), resolve it into an actual directory path, and put that path into `res`.

Function declaration

```
on C
int resolve_target(const char *arg, char *res, size_t nresize)
```

There are 3 arguments:

- `arg` — what the user gave as the target.
- `res`
- `nresize`

8. If `dir` is given, recursively enters directories.

realgar

- Use the new headers.
- Remove your old `read_line()`, `split_line()`, and manual `hop` handling
- Use `lex_line()`.
- Use `parse_tokens()`.
- Let `execute_pipeline(tokens)` dispatch `hop`, `reveal`, etc.
- Initialize both `hop` and `reveal` with `home_c ""`

```

1  // C
2
3  strcpy(username, pw == PW_NAME ? sizeof(username) - 1 :
4  )
5  }
6  else
7  {
8      strcpy(username, "user");
9  }
10
11  if (gethostname(hostname_buf, sizeof(hostname_buf)) != 0)
12  {
13      strcpy(hostname_buf, "host");
14  }
15  }
16
17  /*
18   * Writes the display path into out.
19   * If the current working directory is inside home_dir,
20   * replace the home_dir prefix with ".".
21   */
22  static void get_display_path(char *out, size_t outsize)
23  {
24      char cwd[PATH_MAX];
25
26      if (getcwd(cwd, sizeof(cwd)) == NULL)
27      {
28          fprintf(stderr, "Error: ");
29          return;
30      }
31
32      size_t home_len = strlen(home_dir);
33      if (strcmp(cwd, home_dir) == 0)
34      {
35

```

```

1  // C
2
3  /* EOP (EOF): exit cleanly. */
4  print("on");
5  return 0;
6  }
7
8  The biggest architectural change from your old main.c is this part:
9
10
11  // C
12
13  token_buf* array_strings = malloc(sizeof(token_buf));
14
15  is now:
16
17  // C
18
19  token_list_t *tokens = lex_line(line, &lex_error);
20
21  and this old manual block:
22
23
24  // C
25
26  if (array_strings[0].type == OP_WORD &&
27      strcmp(array_strings[0].value, "hop") == 0)
28  {
29      ...
30      hop(argc, argv, getenv("HOME"));
31  }
32
33  is completely removed.
34

```

```

1  // C
2
3  and this old manual block:
4
5
6  // C
7
8  if (array_strings[0].type == OP_WORD &&
9      strcmp(array_strings[0].value, "hop") == 0)
10  {
11      ...
12      hop(argc, argv, getenv("HOME"));
13  }
14
15  is completely removed.
16
17  Now main.c doesn't need to know how hop() or reveal() work. It just does:
18
19
20  // C
21
22  input
23  {
24      lex_line()
25      {
26          parse_tokens()
27          {
28              execute_pipeline()
29          }
30      }
31      hop() / reveal() / peek() / locate() / external command
32  }
33
34  That is the main architectural change you were making.
35

```

other helper functions that don't need to be visible outside their .c file.

Your parser.h only exposes:

```

1  // C
2
3  int parse_tokens(const token_list_t *list);
4

```

which is exactly what you want externally. [🔗](#) [🔗](#) [🔗](#)

4. Your reveal.c should use static for internal helpers

This connects to what you told me earlier.

These functions are implementation details of reveal.c:

```

1  // C
2
3  cmd_names()
4  resolve_target()
5  reveal_list_dir()
6

```

They don't need to be accessible from main.c or another .c file.

So make them:

```

1  // C
2
3  static int cmd_names();
4

```

but why shld i use static for function declaration? how is it diff?

Yes. The key idea is:

- static on a function definition means: this function can only be used inside this .c file.

It gives the function internal linkage.

Without static:

Suppose parser.c has:

```

1  // C
2
3  int at_end(const parser_t *p)
4  {
5      return p->pos == p->len;
6  }
7

```

This function has external linkage by default.

That means the linker treats at_end as a function that could be used by other .c files.

For example, main.c could theoretically do:

```

1  // C
2
3  at_end(p);
4

```

Conceptually:

```

1  parser.c
2  {
3      static at_end() - PRIVATE
4      static cmd() - PRIVATE
5      static at_end() - PRIVATE
6      static at_end() - PRIVATE
7      static at_end() - PUBLIC
8  }
9

```

Your parser.h only exposes:

```

1  // C
2
3  int parse_tokens(const token_list_t *list);
4

```

which means the intended interface is:

```

1  main.c
2  {
3      calls
4      {
5          parse_tokens()
6          {
7              internally uses
8              {
9                  at_end()
10                 cmd()
11                 at_end()
12                 at_end()
13             }
14         }
15     }
16 }
17

```

```

1  // C
2
3  free(l->items[i]);
4  }
5
6  free(l->items);
7  }
8

```

Then add the read_lines() function:

```

1  // C
2
3  void read_lines(FILE *fp, lines_t *la)
4  {
5      char *line = NULL;
6      size_t cap = 0;
7      ssize_t len;
8
9      int is_terminal = (fp == stdin && isatty(STDIN_FILENO));
10
11      while ((len = getline(&line, &cap, fp)) != -1)
12      {
13          if (len > 0 && line[len - 1] == '\n')
14              line[len - 1] = '\0';
15
16          lines_push(la, strdup(line));
17
18          if (is_terminal)
19              break;
20      }
21
22      free(line);
23  }
24

```

What this does

with:

```

1  // C
2
3  static int at_end(...) { ... }
4

```

The first is a declaration and the second is a definition.

You can have:

```

1  // C
2
3  static int at_end(const parser_t *p) // declaration
4  static int at_end(const parser_t *p) // definition
5  {
6      return p->pos == p->len;
7  }
8

```

But if you're defining the function immediately, you normally just write:

```

1  // C
2
3  static int at_end(const parser_t *p)
4  {
5      return p->pos == p->len;
6  }
7

```

Rule of thumb for your project

Put static on functions that are implementation details of one .c file.

```
line[len-1] = '\0';
lines_push(la, strdup(line));
}

free(line);

for (size_t i = 0; i < la->count / 2; i++)
{
    char *temp = la->items[i];
    la->items[i] = la->items[la->count - 1 - i];
    la->items[la->count - 1 - i] = temp;
}
}

// okay o i tried this for the -r flag of peek, BUT ITS NOT
// COMPATIBLE WITH MY FILE (its giving the error i pasted above).
// Why? can u fix this
```

Yes. The issue is most likely that your existing `peek` code uses a file descriptor (`int fd`), while the function you tried uses a `FILE *fp`.

These are two different ways of accessing files:

```
int fd;
```

goes with:

```
FILE *fp;
```

then this function:

```
void read_lines_backward(FILE *fp, lines_t *la)
```

is not compatible.

Fix: keep your existing `int fd`

Since your original function was:

```
void read_lines_backward(int fd, lines_t *la)
```

you should keep that interface.

But if you want the simpler implementation without `off_t`, `lseek()`, etc., we can convert the file descriptor into a `FILE *` temporarily using `fopen()`:

```
void read_lines_backward(int fd, lines_t *la)
{
    FILE *fp = fopen(fd, "r");
    if (fp == NULL)
        return;

    char *line = NULL;
    size_t cap = 0;
```

```
if (fsize <= 0) return;

char buf[CHUNK];
char *lo = NULL;
size_t lo_len = 0;
off_t pos = fsize;

while (pos > 0) {
    ssize_t toread = ((off_t)CHUNK < pos) ? CHUNK : (size_t)pos;
    pos -= toread;
    lseek(fd, pos, SEEK_SET);
    ssize_t n = read(fd, buf, toread);
    if (n <= 0) break;

    ssize_t end = n;

    for (size_t i = n - 1; i >= 0; i--) {
        if (buf[i] == '\n') {
            size_t seg = end - (i + 1);
            size_t total = (size_t)seg + lo_len;

            char *line = malloc(total + 1);

            if (seg > 0)
                memcpy(line, buf + i + 1, seg);

            if (lo_len > 0)
                memcpy(line + seg, lo, lo_len);

            line[total] = '\0';
            lines_push(la, line);

            free(lo);
            lo = NULL;
```

Why the second one is better for `peek -r`

The key difference is what direction the file is read in.

Your version does:

```
FILE
|
| read line 1
|
| read line 2
|
| read line 3
|
| ...
|
| store everything
|
| reverse everything
```

The second version does:

```
FILE
|
| jump to END
|
| read backwards
|
| find last newline
|
| get last line
|
| find previous newline
|
| get previous line
```

```
char *lo = NULL;
size_t lo_len = 0;
```

You don't have to introduce `FILE *`, `fopen()`, `fclose()`, etc.

What `lo` is doing

This is the trickiest part:

```
char *lo = NULL;
size_t lo_len = 0;
```

`lo` means:

"I found part of a line, but I haven't found the beginning of that line yet."

For example, imagine a chunk ends like:

```
hello wor
```

but the actual line is:

```
hello world
```

The next chunk contains:

```
ld'n
```

When reading backwards, you first encounter

```
memcpy(nlo, buf, end);

if (lo_len > 0)
    memcpy(nlo + end, lo, lo_len);
```

It joins the pieces of a line that crossed a chunk boundary.

So which one should you use?

For a basic implementation where you just need to reverse **every** line:

Your version is much easier.

But for an actual `peek -r` implementation where you want to read the last **N** lines efficiently, I'd recommend the `fd + lseek + read` version.

The important reason is:

It reads from the end of the file instead of reading the entire file and then reversing it.

So I would keep the second version for `peek -r`, especially if your assignment expects `peek -n N -r file` to avoid unnecessarily processing the whole file.

```

        files[file_count++] = arg; // if not a flag then its a file
    }
}

```

okay so like im at this stage rn? But how do i implement the actual peek process? it must be compatible with my architecture. Can u give code or a runthrough and then EXPLAIN it pls?

Yes. You're actually **very close**. Your code already has the two important pieces:

1. `read_lines()` → reads a file/stdin **forward**
2. `read_lines_backward()` → reads a regular file **backward**

What's missing is the part that decides:

- Is the input a file or stdin?
- Is `-r` enabled?
- Is `-n` enabled?
- Which reading function should be used?

```

    }
    else
    {
        (sizeof_t)pos;
    }
}

```

This does **nothing**.

You meant:

```

    }
    else
    {
        (sizeof_t)pos;
    }
}

```

So:

```

    }
    if ((off_t)CHUNK < pos)
    {
        (sizeof_t)pos;
    }
    else
    {
        (sizeof_t)pos;
    }
}

```

Otherwise, `sizeof_t` contains an uninitialized value which can cause very weird errors

3. The simplest way to make peek() work

Since you already have `read_lines()` and `read_lines_backward()`, I'd add one helper function:

```

void process_file(const char *filename, int n_flag, int r_flag, int *line_num)
{
    lines_t la;
    lines_init(&la);

    if (!r_flag)
    {
        int fd = open(filename, O_RDONLY);

        if (fd < 0)
        {
            printf("peek: no such file or directory\n");
            lines_free(&la);
            return;
        }

        read_lines_backward(fd, &la);
        close(fd);
    }
    else
    {
        FILE *fp = fopen(filename, "r");

        if (fp == NULL)
        {
            printf("peek: no such file or directory\n");
            lines_free(&la);
            return;
        }
    }
}

```

```

        fclose(fp);
    }

    for (size_t i = 0; i < la.count; i++)
    {
        if (n_flag)
        {
            printf("hd %s\n", ++(*line_num), la.items[i]);
        }
        else
        {
            printf("%s\n", la.items[i]);
        }
    }

    lines_free(&la);
}

```

Then your `peek()` becomes:

```

void peek(const tokens_list_t *list)
{
    int n_flag = 0;
    int r_flag = 0;

    const char *files[100];
    int file_count = 0;

    for (size_t i = 1; i < list->count; i++)
    {

```

```

        {
            if (n_flag)
            {
                printf("hd %s\n", ++line_num, la.items[i]);
            }
            else
            {
                printf("%s\n", la.items[i]);
            }
        }

        lines_free(&la);
        return;
    }

    /* files were provided */
    for (int i = 0; i < file_count; i++)
    {
        process_file(files[i], n_flag, r_flag, &line_num);
    }
}

```

3. How this works

Suppose you have:

```

file.txt
apple
banana
carrot
dog

```

can u write a function that takes ur arguments argv and and extracts input (<) & output (>, >>) redirection files

Yes. Using the **same logic and structures from your code**, you can make the redirection-extraction part into a separate function:

```

void extract_redirections(char **argv, int argc,
                        char *input_files[], int *num_inputs,
                        output_redir_t output_files[], int *num_outputs)
{
    *num_inputs = 0;
    *num_outputs = 0;

    for (int i = 0; i < argc; i++) {
        if (strcmp(argv[i], "<") == 0) {
            if (i + 1 < argc) {
                input_files[*num_inputs++] = argv[i + 1];
                i++;
            } else {
                printf("catell: invalid syntax\n");
                return;
            }
        }
        else if (strcmp(argv[i], ">") == 0) {

```

int is_append;

}output_redir_t; for outputs?

```

}
}

```

This is the same extraction logic as your existing:

```

    if (list->tokens[i].type == OP_LT)

```

```

    else if (list->tokens[i].type == OP_GT)

```

```

    else if (list->tokens[i].type == OP_GTGT)

```

The difference is that this version receives normal `argv` strings such as:

```

argv = {"cat", "file.txt", "<", "input.txt", ">", "out.txt", NULL}

```

and extracts:

Because in your code, **input redirection only needs one piece of information: the filename**, while **output redirection needs two pieces of information**.

Input: only the path

For `<`:

```

command < input.txt

```

You only need to remember:

```

    "input.txt"

```

So this is enough:

```

    "input.txt"

```

Each element is simply a pointer to the filename.

For example:

```
/* C
input_files[0] = "input.txt";
input_files[1] = "other.txt";
```

There is no difference in behavior between:

```
<input.txt
```

and some other kind of input operator in your current code. Both use `O_RDONLY`.

Output: path + whether to append

For output, you have **two different operators**:

```
> output.txt
```

and

```
/* C
output_files[0].path = "output.txt";
output_files[0].is_append = 1;
```

Then later this line:

```
/* C
int flags = O_WRONLY | O_CREAT |
(output_files[i].is_append ? O_APPEND : O_TRUNC);
```

uses `is_append` to decide what to do.

So basically

Redirection	What needs to be remembered?
< input.txt	"input.txt"
> output.txt	"output.txt" + overwrite
>> output.txt	"output.txt" + append

That's why:

What does the `O_` mean?

`O_RDONLY` is a flag defined in `<fcntl.h>`.

There are three common access modes:

```
/* C
O_RDONLY // read only
O_WRONLY // write only
O_RDWR  // read + write
```

So:

```
/* C
open("input.txt", O_RDONLY);
```

means:

```
input.txt
|
|
open()
|
|
read allowed
write NOT allowed
```

Why not `O_WRONLY`?

`O_RDONLY` tells `open()` how you want to access the file.

In your code:

```
/* C
int fd = open(input_files[i], O_RDONLY);
```

you're saying:

"Open this file so that I can **only read** from it."

Why do we need that?

Because your `<` redirection means:

```
command < input.txt
```

The shell needs to take **data** from `input.txt` and give it to the command's `stdin`.

So the file needs to be opened for reading.

For example:

So `O_RDONLY` is the correct mode.

And notice your output is different

For:

```
cat > output.txt
```

you use:

```
/* C
open(output_files[i].path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

because now the shell needs to **write** into the file.

So the simple rule is:

```
< -> O_RDONLY
> -> O_WRONLY | O_CREAT | O_TRUNC
>> -> O_WRONLY | O_CREAT | O_APPEND
```

Think of `open()` as asking:

"What do I want to do with this file?"

`O_RDONLY` = "I want to read it."

```
/* C
output_files[i].is_append ? O_APPEND : O_TRUNC
```

means:

- if `is_append == 1` → use `O_APPEND`
- if `is_append == 0` → use `O_TRUNC`

Therefore:

For `>`

Your code sets:

```
/* C
output_files[num_outputs].is_append = 0;
```

so:

```
/* C
O_WRONLY | O_CREAT | O_TRUNC
```

`O_TRUNC` means **erase the existing contents** of the file before writing.

For `>>`

Your code sets:

That is exactly what the ternary part is doing.

Today 9:25 AM

```
out_fds[i] = open(output_files[i].path, flags, 0644); but why open
the file with 0644?
```

The `0644` is the **permission mode** given to `open()` when `O_CREAT` creates a new file.

In your code:

```
/* C
out_fds[i] = open(output_files[i].path, flags, 0644);
```

The three arguments are:

```
/* C
open(
  output_files[i].path, // filename
  flags,                // how to open it
  0644                  // permissions if a new file is created
);
```

What does `0644` mean?

```
/* C
output_files[num_outputs].is_append = 0;
```

so:

```
/* C
O_WRONLY | O_CREAT | O_TRUNC
```

`O_TRUNC` means **erase the existing contents** of the file before writing.

For `>>`

Your code sets:

```
/* C
output_files[num_outputs].is_append = 1;
```

so:

```
/* C
O_WRONLY | O_CREAT | O_APPEND
```

`O_APPEND` means **write new data at the end** of the existing file.

So the complete line is essentially:

0644

permissions

The digits mean:

6 4 4

owner group others

And:

6 = read + write
4 = read
4 = read

So:

0644

means:

owner - read + write
group - read
others - read

if we can use ten cwd or skip it (starts with %). Also, if we don't skip the current working dir, look for the executable in it? and explain the code and how it fits into my architecture

Yes. This is the exact part of your architecture that handles the % rule and then checks the current working directory.

```
on C
if (raw_cmd[0] == '%') {
    skip_cwd = 1;
    cmd_name = raw_cmd + 1;
    argv[0] = (char *)cmd_name;
}

char *exec_path = NULL;

/* 2. Check CWD unless skipped by '%' */
if (!skip_cwd) {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        char cwd_candidate[PATH_MAX + 512];
        sprintf(cwd_candidate, "%s/%s", cwd, cmd_name);
        if (executable(cwd_candidate)) {
            exec_path = strdup(cwd_candidate);
        }
    }
}
```

1. Check whether the command starts with %

Flowchart:

```
graph TD
    Start(( )) --> Check{ }
    Check -- YES --> SkipCMD[skip CMD]
    Check -- NO --> SearchCMD[search CMD]
    SkipCMD --> PathSearch[PATH search]
    SearchCMD --> ExecutableFound{executable found?}
    ExecutableFound -- YES --> ExecPath[exec_path]
    ExecutableFound -- NO --> Continue[continue]
    ExecPath --> PathSearch
```

And this fits directly into the code you already have, before your PATH search:

```
on C
if (raw_cmd[0] == '%') {
    skip_cwd = 1;
    cmd_name = raw_cmd + 1;
    argv[0] = (char *)cmd_name;
}

char *exec_path = NULL;

/* Check CWD unless skipped by '%' */
if (!skip_cwd) {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        char cwd_candidate[PATH_MAX + 512];
        sprintf(cwd_candidate, "%s/%s", cwd, cmd_name);
        if (executable(cwd_candidate)) {
            exec_path = strdup(cwd_candidate);
        }
    }
}
```

So if the user enters:

%myprogram

then:

```
on C
raw_cmd[0] == '%'
```

is true.

2. Tell the program to skip CWD

```
on C
skip_cwd = 1;
```

This records:

Don't search the current working directory.

So % acts as your indicator for skip CWD.

3. Remove % from the command name

okay but like wat of teh file ISNT in the cwd? wat o do next??

If the executable isn't in the CWD, your code should move on to the PATH search.

That's exactly what this condition does:

```
on C
if (exec_path == NULL) {
```

If the CWD search didn't find anything, exec_path is still NULL, so you enter this block.

Then your code gets the PATH:

```
on C
const char *env_path = getenv("PATH");
```

For example, PATH might contain:

/usr/local/bin:/usr/bin:/bin

Then:

```
on C
char *path_copy = strdup(env_path);
```

```
on C
...
/usr/local/bin/mycommand
/usr/bin/mycommand
/bin/mycommand
...
```

So it checks something like:

For each one, you construct:

```
on C
sprintf(candidate, "%s/%s", dir, cmd_name);
```

and check:

```
on C
if (executable(candidate)) {
    exec_path = strdup(candidate);
    break;
}
```

So your overall architecture is:

```
command
|
| starts with %?
|
| YES -- skip CWD -- PATH search
|
| NO
|
| search CWD
|
| found?
|
| YES -- NO
|
| exec_path PATH search
```

command

starts with %?

YES -- skip CWD -- PATH search

NO

search CWD

found?

YES -- NO

exec_path PATH search

And the PATH-search portion you already have is:

```
on C
if (exec_path == NULL) {
    const char *env_path = getenv("PATH");
    if (env_path != NULL && strlen(env_path) > 0) {
        char *path_copy = strdup(env_path);
        if (path_copy != NULL) {
            char *saveptr = NULL;
            char *dir = strtok_r(path_copy, ":", &saveptr);
            while (dir != NULL) {
                char candidate[PATH_MAX + 512];
                sprintf(candidate, "%s/%s", dir, cmd_name);
                if (executable(candidate)) {
                    exec_path = strdup(candidate);
                    break;
                }
                dir = strtok_r(NULL, ":", &saveptr);
            }
        }
    }
}
```

okay so, u have my existing code now? NOW, i want to be able to read from a file in the < case. But i need to read from multiple files if theres are cascading < HOW do i implement that BASED PON MY CURRENT CODE STRUCTURE

Yes. Based on your current structure, you've already implemented the multiple-< case using a pipe and a feeder process.

The relevant part is:

```
on C
if (num_inputs == 1) {
    int in_fd = open(input_files[0], O_RDONLY);
    if (in_fd == 0) {
        dup2(in_fd, STDIN_FILENO);
        close(in_fd);
    }
} else if (num_inputs > 1) {
    int pfd[2];
    pipe(pfd);
    ...
}
```

```
waitpid(feeder, NULL, 0);
}
}
```

How this works

Suppose the command is:

```
cat < file1 < file2 < file3
```

Your extraction code gives you:

```
num_inputs = 3
input_files[0] = file1
input_files[1] = file2
input_files[2] = file3
```

Then:

```
// C
else if (num_inputs > 1)
```

creates a pipe:

```
execv(exec_path, argv);
```

runs `cat`, `cat` reads from `stdin`, which is now connected to the pipe.

One important detail

Your code currently does:

```
// C
waitpid(feeder, NULL, 0);
```

before `execv()` runs.

That means the parent/command process waits for the feeder to finish writing before executing the command. For multiple input files, this can potentially cause a **pipe deadlock** if the combined input is larger than the pipe's capacity, because the feeder can block waiting for the pipe to be read while the command hasn't started reading yet.

But structurally, your approach is:

open each `<` file $\rightarrow$ read each file $\rightarrow$ write everything into one pipe $\rightarrow$ connect pipe to `stdin` $\rightarrow$ execute the command.

🔍 🔄 🗑️ ...

```
// C
for (int i = 0; i < num_inputs; i++) {
```

For each file:

```
// C
int f = open(input_files[i], O_RDONLY);
```

Then it reads chunks:

```
// C
while ((bytes = read(f, buf, sizeof(buf))) > 0)
```

and writes those chunks into the pipe:

```
// C
ssize_t w = write(pfd[1], buf, written);
```

So the contents effectively become:

```
file1 contents
+
file2 contents
```

Yes. Your existing output-redirection code **already applies to both `>` and `>>`** because `out_fds[]` was opened earlier using the `is_append` flag.

So you only need to keep this code essentially unchanged:

```
// C
if (num_outputs == 1) {
    dup2(out_fds[0], STDOUT_FILENO);
    close(out_fds[0]);
} else if (num_outputs > 1) {
    int out_pfd[2];
    if (pipe(out_pfd) == 0) {
        pid_t writer = fork();
        if (writer == 0) {
            close(out_pfd[1]);
            char buf[4096];
```

For one output

```
echo hello > a.txt
```

becomes:

```
echo
├─ stdout
└─ dup2(out_fds[0], STDOUT_FILENO)
├─
└─ a.txt
```

For:

```
echo hello >> a.txt
```

the exact same redirection code is used, but `a.txt` was opened with `O_APPEND`, so the output goes to the **end** instead of replacing the contents.

For multiple outputs

```
echo hello > a.txt > b.txt
```

your pipe code does:

```
}
}

The important thing is how out_fds[] was created earlier:
```

```
// C
int flags = O_WRONLY | O_CREAT;
if (output_files[i].is_append) {
    flags = flags | O_APPEND;
} else {
    flags = flags | O_TRUNC;
}
out_fds[i] = open(output_files[i].path, flags, 0644);
```

Your extraction code sets:

```
// C
output_files[num_outputs].is_append = 0;
```

for:

```
>
```

For:

```
echo hello >> a.txt
```

the exact same redirection code is used, but `a.txt` was opened with `O_APPEND`, so the output goes to the **end** instead of replacing the contents.

For multiple outputs

```
echo hello > a.txt > b.txt
```

your pipe code does:

```
echo ── stdout ── pipe ──
      │          │      │
      └─ a.txt   └─ b.txt
```

The `writer` process reads from the pipe and loops over:

```
// C
for (int i = 0; i < num_outputs; i++)
```

so the same output is written to every output file.

Thus, **minimal change** = essentially no change to this output-redirection block. The `>` vs `>>` behavior is determined when you `name()` the files, not when you `dup2()` / `write()` to them.

```
else if (strcmp(cmd_name, "peek")==0)
{
    peek(list);
    return;
}
else if (strcmp(cmd_name, "locate")==0)
{
    locate(list);
    return;
}
}
execute_cmd(list);
return;
}
```

okay so my code finds the number of stages in the command and then executes a single stage setup. Wat do i do for a multi stage case? like can u add a code block THAT IS COMPATIBLE with mine and implements teh same

Yes. Since your current code already handles the **single-stage case**, the next thing is to add an `else` for `num_stages > 1`.

Based on the pipeline architecture you already showed me, you can add the **multi-stage setup** after your `if (num_stages == 1)` block:

```
if (list->tokens[i].type == OP_PIPE) {
    if (stage_count < 256) {
        stages[stage_count].tokens = &list->tokens[start_idx];
        stages[stage_count].count = i - start_idx;
        stage_count++;
    }

    start_idx = i + 1;
}

if (start_idx < list->count && stage_count < 256) {
    if (list->tokens[start_idx].type != OP_WORD &&
        list->tokens[start_idx].type != OP_ARG) {
        stages[stage_count].tokens = &list->tokens[start_idx];
        stages[stage_count].count = list->count - start_idx;
        stage_count++;
    }
}

if (stage_count < num_stages) {
    printf("cshell: invalid syntax\n");
    return;
}

for (int i = 0; i < stage_count; i++) {
    if (stages[i].count == 0) {
        printf("cshell: invalid syntax\n");
        return;
    }
}
```

```
for (int i = 0; i < stage_count - 1; i++) {
    if (pipe(pipes[i]) < 0) {
        perror("cshell: pipe failed");
        return;
    }
}
```

How this fits your existing code

Your current flow is:

```
on C
int num_stages = 1;
```

Then you count | :

```
on C
if (list->tokens[i].type == OP_PIPE)
{
    num_stages++;
}
```

So:

```
cat file
```

```
on C
execute_cmd(list);

But:
cat file | grep hello | wc

gives:
num_stages = 3

so execution goes into:
on C
else
{
    ...
}

The added code first splits the command into:
stage 0 - cat file
stage 1 - grep hello
stage 2 - wc
```

ones i have the stage list. CAAN U LOOK at my lt, gt and gtgt and if (num_stages==1)

if (list->count>0&&list->tokens[0].type==OP_WORD) //is forst command a word(cat, exec, ocate et.)

const char *cmd_name = list->tokens[0].text;

if (strcmp(cmd_name, "hop")==0)

{
 hop(list);
 return;

}
else if (strcmp(cmd_name, "reveal")==0)

{
 reveal(list);
 return;

}
else if (strcmp(cmd_name, "peek")==0)

{
 peek(list);
 return;

}
else if (strcmp(cmd_name, "locate")==0)

{
 locate(list);
 return;

}
...
}

```
reveal(list);
return;
}
else if (strcmp(cmd_name, "peek")==0)
{
    peek(list);
    return;
}
else if (strcmp(cmd_name, "locate")==0)
{
    locate(list);
    return;
}
}
execute_cmd(list);
return; to create a code block that works with my code and is
compatible with my data structs and does the same
```

Yes. Looking at your existing OP_LT, OP_GT, OP_GTGT handling and your current single-stage dispatch, the multi-stage part should create one child per stage and let each child handle its own explicit redirection.

This is compatible with your existing stage_t, token_list_t, OP_LT, OP_GT, OP_GTGT, pipes, and built-ins:

```
on C
pid_t pids[256];
```

For something like:

```
cat file | grep hello | wc
```

you have:

```
stage 0 - cat
cat - stage 1 - grep
grep - stage 2 - wc
```

and your loop:

```
on C
for (int i = 0; i < stage_count; i++)
```

creates:

```
child 0 - cat
child 1 - grep
child 2 - wc
```

The important part with <, > and >>

You specifically need these checks because an explicit redirection should take priority over the pipeline.

For input:

How it fits your current code

Your current structure is:

```
on C
if (num_stages == 1)
{
    ...
    execute_cmd(list);
    return;
}
```

So you can have:

```
on C
if (num_stages == 1)
{
    /* your existing single-stage code */
}
else
{
    /* create stages */
    /* create pipes */
    /* create children */
}
```

For something like:

```
/* If there is no "<" or ">", send output to next pipe */
if (token_arg_idx == 0 && stage_count == 1) {
    buildtoken_list();
}

/* 2. Close all pipe file descriptors on this child */
for (int i = 0; i < stage_count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* 3. Create a token_list_t for this stage */
token_list_t stage_list = {
    .tokens = stages[i].tokens,
    .count = stages[i].count
};

/* 4. Handle built-in commands */
if (stage_list.count == 0) {
    stage_list.tokens[0].type == OP_ARG) {
        const char *cmd_name = stage_list.tokens[0].text;

        if (strcmp(cmd_name, "hop") == 0) {
            hop(stage_list);
            exit(0);
        }
        else if (strcmp(cmd_name, "reveal") == 0) {
            reveal(stage_list);
            exit(0);
        }
        else if (strcmp(cmd_name, "peek") == 0) {
            peek(stage_list);
            exit(0);
        }
        else if (strcmp(cmd_name, "locate") == 0) {
            locate(stage_list);
            exit(0);
        }
    }
}
```



```
C
if (b->frequency != a->frequency)
    return b->frequency - a->frequency;
```

So frequency is always the first priority.

For example:

```
A:
frequency = 5
last access = 1 minute ago

B:
frequency = 8
last access = 2 days ago
```

Your ranking is:

```
B > A
```

because:

```
B > 5
```

The fact that A was accessed one minute ago doesn't matter.



i didnt understand this part of the code? can u tell me what exactly i have to implement , using examples

Based on your screenshots, this assignment asks you to implement **Background Execution (&)** in your custom shell. This means your shell should be able to run commands asynchronously so that you don't have to wait for one command to finish before typing the next one.

Here is a comprehensive breakdown of what you need to implement, broken down by core components, along with an explanation of the provided example.

Core Implementation Checklist

1. Forking and Launching (& parsing)

- o **Fork & Go:** When a command ends with an ampersand (&), your shell should `fork()` a child process to execute it. Unlike foreground commands, **do not call** `wait()` on it immediately.

- o **Multiple Commands:** If a user enters multiple commands separated by a semicolon (;), your shell should execute them sequentially.

- o **Job & Process IDs:** Right after launching a background job, print its assigned job number and PID in the format `[job_number] process_id` **before** any output from the command itself appears.
- o **Monotonic Job Numbers:** Job numbers start at 1 and must continuously increase globally across your shell session. They **never** reset or get reused, even if a previous job finishes.
- o **Prompt Return:** After launching all background jobs on a line, your shell must immediately reprint the prompt and wait for the next user input.

2. Handling Completion via SIGCHLD

- o **Signal Handler:** Install a `SIGCHLD` signal handler to catch when your background child processes finish executing.
- o **Non-Blocking Reaping:** Inside your `SIGCHLD` handler, use `waitpid(-1, &status, WNOHANG)` in a loop. Using `WNOHANG` ensures the shell never blocks or freezes waiting for processes.
- o **Exit Status Reporting:**
 - o If `WIFEXITED(status)` is true, print:
`<command_name> with pid <pid> exited normally .`
 - o If `WIFSIGNALED(status)` is true, print:
`<command_name> with pid <pid> exited abnormally .`
- o **Timing of Messages:**
 - o If the shell is currently sitting at the prompt waiting for user input, print the termination message immediately.

if i dont call wait on child process immediately, wont that lead to zombies?

Yes, exactly! If you fork a background process and *never* call `wait()` on it anywhere, it will indeed become a zombie process when it finishes, lingering uselessly in the system's process table.

However, that is precisely why requirements #6 and #7 in your assignment are there. You avoid zombies without blocking your shell by using an asynchronous mechanism:

How You Prevent Zombies Without Blocking

1. **The Signal (`SIGCHLD`):** When any child process finishes execution, the operating system automatically sends a `SIGCHLD` signal to your parent shell process.
2. **The Handler:** Your shell catches this signal using a custom signal handler function you write.
3. **The Reaping (`waitpid + WNOHANG`):** Inside that signal handler, you call `waitpid(-1, &status, WNOHANG)`.
 - o The `WNOHANG` flag is the secret sauce here: it tells `waitpid` to check for finished children and **reap them immediately**, but if no child has finished yet, it returns `0`.

Next
Github do i do this in windows (do i make in home dir or project dir?)

Yep — on **Windows**, the easiest route is the **Distrobox/container approach**, but there's an even simpler option if you have **WSL2**: use Ubuntu inside WSL. You do **not** want to install/build xv6 directly in normal Windows PowerShell/CMD.

Where do you run `make`?

Inside the **xv6** project directory.

For example:

```
~/xv6-riscv/  
├─ Makefile  
├─ kernel/  
├─ user/  
├─ README  
└─ ...
```

You `cd` into `xv6-riscv` and then run:

```
❯ Bash  
make qemu
```

Not from your home directory.

Everything from your course's **Linux instructions** can now be done inside this terminal.

2. Install the dependencies

Inside Ubuntu:

```
❯ Bash  
sudo apt update  
sudo apt install git make qemu-system-misc gdb-multiarch
```

Then you'll need the RISC-V compiler:

```
❯ Bash  
sudo apt install gcc-riscv64-linux-gnu
```

You can check that it worked:

```
❯ Bash  
riscv64-linux-gnu-gcc --version
```

and:

```
❯ Bash  
qemu-system-riscv64 --version
```

Important distinction

There are **two shells** involved:

Ubuntu/WSL shell

Looks something like:

```
username@computer:~/xv6-riscv$
```

Here you run:

```
❯ Bash  
make qemu  
make clean  
git commands  
gdb
```

xv6 shell

Looks like:

```
$
```

Here you run xv6 programs:

```
$ ls  
$ cat README  
$ echo hello
```

When you want to leave xv6:



What is QEMU?

QEMU is a **computer emulator**.

Your actual laptop probably has an x86-64 or ARM CPU. xv6 in your course targets **RISC-V**.

So QEMU pretends to be a RISC-V computer:

“Hey xv6, pretend I’m an actual RISC-V machine. Here’s a CPU, RAM, disk, etc.”

xv6 then boots on that **fake** machine.

What does `make` do?

When you run `make`:

The `qemu` target in the Makefile tells QEMU:

“Start a RISC-V virtual machine and boot this xv6 kernel.”

So when you run:

```
Bash
make qemu
```

it’s essentially:

```
compile xv6
+
+ create xv6 kernel + filesystem
+
+ start QEMU
+
+ QEMU pretends to be a RISC-V computer
+
+ RISC-V computer boots xv6
+
+ xv6 starts its shell
+
$
```

That `$` you’re seeing is **xv6 actually running**, not just some simulation of a shell.

And this is why your course has you modify xv6: when you change something in `kernel/`, you can rebuild it and then **actually boot your modified operating system** inside QEMU without risking your

```
// We don't actually need the signal number.
// Casting it to void prevents a compiler warning.

int saved_errno = errno;
// Save errno.
//
// Signal handlers can call functions that modify errno.
// We don't want this handler to accidentally change errno
// for whatever code was interrupted.

int status;
// waitpid() will put information about the child here.

for (int i = 0; i < MAX_BQ_JOBS; i++)
{
    // Look through every possible job slot.

    if (jobs_list[i].active && !jobs_list[i].finished)
    {
        // Only examine jobs that:
        //
        // active = currently being tracked
        // finished = not already completely finished

        for (int k = 0; k < jobs_list[i].num_procs; k++)
        {
            // A job can contain multiple processes.
            //
            // For example:
            //
            //   ...
```

```

            // The process has terminated.
            //
            // WIFEXITED:
            //   Normal exit, e.g. exit(0)
            //
            // WIFSIGNALED:
            //   Killed by a signal, e.g. SIGTERM.

            jobs_list[i].procs[k].state = PROC_EXITED;
            // Mark this process as exited.

            if (pid == jobs_list[i].pgid)
            {
                // pgid = process-group ID.
                //
                // This code treats the process whose PID
                // equals the PGID as the representative/
                // leader of the job.

                jobs_list[i].finished = 1;
                // Mark the entire job as finished.

                jobs_list[i].exit_status = status;
                // Save the raw waitpid() status.

                jobs_list[i].is_signaled = WIFSIGNALED(status);
                // Remember whether the process died
                // ...
```

im using fedors. Give me the steps for that pls

Yep — on **Fedora**, the package names are a little different. Follow these steps in order.

1. Install the required packages

Open a terminal and run:

```
❯ Bash
sudo dnf update
```

Then install the basic tools:

```
❯ Bash
sudo dnf install git make gcc gcc-c++ qemu-system-riscv gdb
```

Now install the RISC-V cross-compiler:

```
❯ Bash
sudo dnf install gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

If Fedora says that `gcc-riscv64-linux-gnu` isn't available, tell me the exact error — Fedora versions/repos can differ, and we'll use the appropriate package/source.

2. Check that everything is installed

Run:

```
❯ Bash
qemu-system-riscv64 --version
```

Then:

```
❯ Bash
riscv64-linux-gnu-gcc --version
```

Then:

```
❯ Bash
gdb --version
```

And:

```
❯ Bash
make --version
```

All four should print version information.

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

That `$` means **xv6 is running successfully**.

Try:

```
$ ls
```

You should see xv6's files/programs.

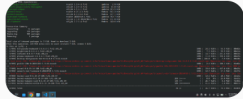
5. Exit xv6

To quit QEMU:

Press `Ctrl+A`, release, then press `X`.

So:

```
Ctrl + A
then
X
```



its been going on for waay too long. Is it an issue if i ctrl c the process beech mein

Yes — you can press `Ctrl+C`. It won't damage Fedora.

Your screenshot shows that `dnf` is downloading **~2 GiB** and **updating 958 packages**, which is far more than you need just to install `xv6` dependencies. The command `sudo dnf update` is what caused this huge operation.

Do this now

Press:

```
Ctrl+C
```

It will stop the current `dnf` operation. You may see something like:

```
^C
```

```
riscv64-unknown-elf-gcc (GCC) ...
```

you're good. That's the bare-metal/newlib-style compiler `xv6` expects. [🔗 GitHub](#)

If only:

```
❯ Bash
```

```
riscv64-linux-gnu-gcc
```

works, that's a **Linux-targeting RISC-V** compiler. Some `xv6` versions/course setups use this, but the current MIT `xv6-riscv` README specifically asks for the RISC-V newlib toolchain. [🔗 GitHub](#)

If `riscv64-unknown-elf-gcc` is NOT found

On Fedora, don't immediately build the entire RISC-V toolchain from source — that can take a **long time** and **several GB of disk space**. The upstream toolchain documentation says a source build can require around 8 GB of disk space. [🔗 GitHub](#)

First check what Fedora has available:

```
❯ Bash
```

```
dnf search riscv64
```

You can also check specifically:

```
❯ Bash
```

Step 2 — Remove the nested Git repository

Go to your home directory:

```
❯ Bash
```

```
cd ~
```

Clone the toolchain:

```
❯ Bash
```

```
git clone https://github.com/riscv-collab/riscv-gnu-toolchain.git
```

Then:

```
❯ Bash
```

```
cd riscv-gnu-toolchain
```

Step 3 — Create an installation directory

We'll install it in:

```
/opt/riscv
```

Create it:

```
❯ Bash
```

today 5:47 AM

no error is due to xv6 beng a diff mit cloned repo

Ahh yes — if `xv6` itself is a **separate Git repo** cloned from MIT, then `git add / commit` may be trying to treat it as a **nested Git repository**, rather than committing its files into *your* repo.

If your structure is something like:

```
mini-project1/
├── .git/                -- YOUR repo
├── xv6-riscv/
│   ├── .git/           -- MIT xv6 repo
│   ├── Makefile
│   └── ...
```

then `xv6-riscv` is a **nested Git repository**.

If you want the `xv6` files to be part of YOUR repo

From your **main repo** directory:

```
❯ Bash
```

```
cd ~/mini-project1
```

First remove the nested Git metadata:

```
❯ Bash
```

⌵

Bash

git status

If you see something like:

⌵

Bash

modified: xv6-riscv (modified content)

or:

⌵

Bash

new file: xv6-riscv

send me the output of:

⌵

Bash

git status
git ls-files --stage xv6-riscv

and I can tell you **exactly** what command to run without messing up your repo.

⌵

🔗

🔄

📄

⋮

Today 5:49 AM

pritya@fedora:~/mini-project1_2025114007\$ git add xv6/Makefile
fatal: Pathspec 'xv6/Makefile' is in submodule 'xv6'
ypriya@fedora:~/mini-project1_2025114007\$ still iddue

⌵

Bash

cd ~/mini-project1_2025114007

2. Remove the submodule registration

⌵

Bash

git submodule deinit -f xv6
git rm -f xv6

Don't worry: `git rm` removes the submodule from the parent repo's index. We'll put the xv6 files back as normal files.

3. Remove the old submodule metadata

⌵

Bash

rm -rf .git/modules/xv6

4. Clone/copy xv6 back as a normal directory

If you still have the xv6 directory somewhere else, copy it back:

⌵

Bash

cp -r /path/to/your/xv6 ./xv6

⌵

Bash

git clone <YOUR_XV6_REPO_URL> xv6

Important: after cloning, remove its `.git`:

⌵

Bash

rm -rf xv6/.git

5. Now check

⌵

Bash

git status

You should see `xv6/` as regular files rather than a submodule.

Then:

⌵

Bash

git add xv6/Makefile
git commit -m "Add scheduler compiler flags"
git push origin main

⚠️ But before doing all this

Since you already have `xv6` inside your repo, there's a good chance we can fix this without deleting/re-cloning anything.

⌵

Bash

git commit -m "Add scheduler compiler flags"
git push origin main

If `git rm --cached xv6` also gives an error

Stop there and paste the output of:

⌵

Bash

git ls-files --stage xv6
git status
cat .gitmodules 2>/dev/null

I'll work from your exact Git state rather than having you run commands that might remove something you need.

⌵

🔗

🔄

📄

⋮

Today 5:50 AM

fatal: could not lookup name for submodule 'xv6'
ypriya@fedora:~/mini-project1_2025114007\$ git ls-files --stage xv6
160000 9e3161a9abf5f1ea402562d1874caf5c4926597 0 xv6
ypriya@fedora:~/mini-project1_2025114007\$ git rm --cached xv6
error: the following file has staged content different from both the
file and the HEAD:
xv6

⌵

Bash

git commit -m "Add scheduler compiler flags"
git push origin main

Now remove the nested Git metadata:

⌵

Bash

rm -rf xv6/.git
rm -rf .git/modules/xv6

Then check:

⌵

Bash

git status

You should now see `xv6/` as a bunch of **new files**, rather than as a submodule.

Then add your Makefile

If you only want the Makefile:

⌵

Bash

git add xv6/Makefile
git commit -m "Add scheduler compiler flags"
git push origin main

If you want all of `xv6` tracked by your repo:

⌵

Bash

```
Untracked files:
(use "git add <file>..." to include in what will be committed)
mlfq_timeline.png
plt.py
xv6/

no changes added to commit (use "git add" and/or "git commit -a")
ypriya@fedora:~/mini-project1_2025114007$ thsi okay?
```

Yes!  This is exactly what we wanted.

Your `xv6/` is now showing under:

```
Untracked files:
xv6/
```

That means it is **no longer a submodule**. Git can now track the files inside it normally.

Now add the Makefile

If you only want to commit the scheduler Makefile change:

```
❯ Bash
git add xv6/Makefile
```

Then check:

```
. .
```

Yep — I see exactly what happened. 

Nothing got committed. This line tells us:

```
no changes added to commit
```

And therefore:

```
git push
Everything up-to-date
```

Is correct — there was no new commit to push.

Why `git add xv6/Makefile` **didn't work**

Your `xv6/` is now a normal directory, but the **Makefile is apparently already tracked by the `xv6` repository's state in a way that needs checking** — or, more likely, the `Makefile` you edited isn't actually different from the version you're trying to add.

Let's check what's happening.

Run these 3 commands:

```
❯ Bash
git status --short xv6
```

```
drwxr-xr-x. 1 ypriya ypriya 12 Sep 13 05:38 gk
-rw-r--r--. 1 ypriya ypriya 22 Sep 12 21:46 HEAD
drwxr-xr-x. 1 ypriya ypriya 556 Sep 12 21:43 hooks
-rw-r--r--. 1 ypriya ypriya 6791 Sep 12 21:46 index
drwxr-xr-x. 1 ypriya ypriya 14 Sep 12 21:43 info
drwxr-xr-x. 1 ypriya ypriya 16 Sep 12 21:46 logs
drwxr-xr-x. 1 ypriya ypriya 16 Sep 12 21:43 objects
-rw-r--r--. 1 ypriya ypriya 326 Sep 12 21:46 packed-refs
drwxr-xr-x. 1 ypriya ypriya 32 Sep 12 21:46 refs
ypriya@fedora:~/mini-project1_2025114007$
```

AHA. We found the issue. 

`xv6/.git` **still exists**, so `xv6` is still its own Git repository. That's why your parent repo treats the whole `xv6/` directory as untracked rather than letting you commit individual files normally.

You need to remove **`xv6`'s internal Git metadata**, not the `xv6` files.

Do this:

You're already in:

```
~/mini-project1_2025114007
```

Run:

```
❯ Bash
```

```
❯ Bash
git diff -- xv6/Makefile
```

```
❯ Bash
ls -la xv6/.git
```

Paste the output.

Also: your `rm -rf` did NOT necessarily break `xv6`

Removing:

```
❯ Bash
xv6/.git
```

only removes the **Git history/metadata inside `xv6`**. It does **not** remove `xv6`'s source code.

The important thing now is figuring out why Git isn't seeing your `Makefile` as a change.

Don't run `git add .` yet, because that could stage:

```
README.md
mlfq_timeline.png
plt.py
```

which you may not want in this commit.

```
. .
```

```
Bash
git add xv6/Makefile
```

Check:

```
Bash
git status
```

You should now see:

```
Changes to be committed:
  new file:   xv6/Makefile
```

Then:

```
Bash
git commit -m "Modify xv6 Makefile for scheduler"
git push origin main
```

BUT one important thing

Because `xv6/` is currently completely untracked, your parent repo does **not** have any `xv6` files yet.

So if your assignment requires the **entire `xv6` source tree to be inside your GitHub repo**, you actually want:

```
Bash
```

The highlighted section

```
C
if (which_dev == 2) {
```

- `devintr()` returns `2` when the interrupt was caused by the **timer**.
- So everything inside this `if` runs **once per timer interrupt**.

```
C
p->rtime++;
```

- `p` is the **currently running process**.
- `rtime` = running time.
- Every timer tick, increase its running time by `1`.

```
C
for (struct proc *rp = proc; rp < &proc[NPROC]; rp++) {
```

- Go through **every process** in the process table.
- `rp` means "reference process".
- `proc` is the first process.
- `&proc[NPROC]` is the address just after the last process.

```
C
```

```
C
#ifdef SCHEDULER_FIFO
```

- Compile this section if `SCHEDULER_FIFO` was defined during compilation.

```
C
// Non-preemptive FIFO: do not yield on timer interrupt
```

- FIFO here is **non-preemptive**.
- The currently running process keeps the CPU.
- Therefore, no `yield()` is called when the timer fires.

```
C
#else defined(SCHEDULER_MLFQ)
```

- Otherwise, if `SCHEDULER_MLFQ` is defined, use the **MLFQ scheduler**.

MLFQ section

```
C
static const int slice_ticks[] = {1, 4, 8, 16};
```

This defines the time quantum for each priority queue:

Priority	Time slice
1	1
2	4
3	8
4	16

```
C
if (p->priority < 3) {
```

- Check whether it can be moved to a lower-priority queue.
- Priority `3` is the lowest priority in this implementation.

```
C
p->priority++;
```

Move the process down one priority level.

For example:

```
C
priority 0 - priority 1
priority 1 - priority 2
priority 2 - priority 3
```

```
C
}
```

End of the priority check.

```
C
yield();
```

Give up the CPU!

"I should give up the CPU because someone more important is waiting."

```
on C
    break;
```

- Stop searching through the processes.
- We already found a higher-priority runnable process.

```
on C
}
```

- End of the condition.

```
on C
}
```

- Finished checking processes for this priority level.

```
on C
    if (should_yield) break;
```

- If a higher-priority process was found, stop checking other priority levels.

```
on C
}
```

"I should give up the CPU because someone more important is waiting."

```
on C
    break;
```

- Stop searching through the processes.
- We already found a higher-priority runnable process.

```
on C
}
```

- End of the condition.

```
on C
}
```

- Finished checking processes for this priority level.

```
on C
    if (should_yield) break;
```

- If a higher-priority process was found, stop checking other priority levels.

```
on C
}
```

1. allocproc() — initialize all scheduling statistics

```
on C
p->priority = 0;
p->ticks_used = 0;
```

p->priority = 0;

- Every newly created process starts in **priority queue 0**.
- In your MLFQ implementation, queue **0** is the **highest-priority queue**.
- So every process gets the same initial priority.

p->ticks_used = 0;

- The process hasn't consumed any CPU time in its current queue yet.
- This counter is incremented by the timer interrupt code:

```
on C
p->ticks_used++;
```

So a new process starts as:

```
priority = 0
ticks_used = 0
```

Then it can use its queue-0 time slice of 1 tick.

Process timing variables

```
on C
acquire(&tickslock);
p->ctime = ticks;
release(&tickslock);
```

acquire(&tickslock);

- `ticks` is a global timer counter.
- Since `ticks` can be modified by the timer interrupt, lock it before reading it.

p->ctime = ticks;

- Save the current global tick count as the process's **creation time**.
- `ctime` tells you approximately **when the process was created**.

release(&tickslock);

- Finished accessing `ticks`, so release the lock.

This is particularly important for your FIFO scheduler because it does:

```
on C
if (earliest == 0 || p->ctime < earliest->ctime)
```

So FIFO uses `ctime` to determine:

Which **RUNNABLE** process was created earlier?

So these two variables work together:

```
rtime - time actually using CPU
wtime - time ready but waiting for CPU
```

```
on C
p->etime = 0;
```

- Initialize **exit time** to zero.
- While the process is alive, it hasn't exited, so there is no exit timestamp yet.

When the process exits, `kexit()` sets:

```
on C
p->etime = ticks;
```

So you get:

```
ctime = process creation time
rtime = CPU time
wtime = waiting time
etime = process exit time
```

```
response time = first_run - ctime
```

2. `kexit()` — record when a process finishes

Your code:

```
in C
acquire(&p->lock);
acquire(&tickslock);
p->etime = ticks;
release(&tickslock);

p->state = status;
p->state = ZOMBIE;
```

The important addition is:

```
in C
acquire(&tickslock);
p->etime = ticks;
release(&tickslock);
```

Why?

When a process exits, you want to remember the exact timer tick at which it exited.

```
in C
p->state = ZOMBIE;
```

means:

| The process has finished, but its parent hasn't collected its information yet.

Your `waitx()` will later collect its timing information before `freeproc()` destroys the process structure. [Plotted code](#)

3. `waitx()` — return `rtime` and `wtime` to the parent

This is the main purpose of your `waitx()` addition.

Normal xv6:

```
in C
wait(...)
```

basically tells the parent:

| "Your child exited; here's its PID/status."

Your `waitx()` additionally tells the parent:

| "Here's how much CPU time and waiting time your child used."

Find the parent's children

```
// C
acquire(&wait_lock);
```

- Lock the wait-related process information.

```
// C
for (;;) {
```

- Keep searching until a child exits or the parent can no longer wait.

```
// C
for (pp = proc; pp < &proc[NPROC]; pp++) {
```

- Scan every process.

```
// C
if (pp->parent == p) {
```

- Check whether `pp` is a child of the current process.

Find an exited child

```
// C
```

- Save the child's PID.

```
// C
if (addr != 0 &&
    copyout(...,
            (char *)japp->state,
            sizeof(pp->state)) < 0)
```

- Copy the child's exit status from kernel memory into the parent's user memory.

The important additions

Copy `ptime`

```
// C
if (ptime != 0 &&
    copyout(p->pagetable, p->sz, ptime,
            (char *)japp->ptime,
            sizeof(pp->ptime)) < 0)
```

This means:

└ If the parent supplied an address for `ptime`, copy the child's `ptime` there.

So the parent can learn:

```
release(&pp->lock);
release(&wait_lock);
return pid;
```

- Release locks and return the child's PID.

So `waitx()` is basically:

```
Parent calls waitx()
├ Find child
├ Wait until child becomes ZOMBIE
├ Get:
│   ├── exit status
│   ├── ptime
│   └── utime
├ Copy values to parent
├ Free child
└ Return PID
```

Your implementation is visible in the file at lines 436–490. [Paste code](#)

4. FIFO scheduler — why `ctime` and `first_run` are used

Your FIFO scheduler searches all `RUNNABLE` processes:

```
first_run -- record when it actually started running
```

Your FIFO implementation is at lines 498–540. [Paste code](#)

5. MLFQ scheduler — priority and global boost

This is where:

```
// C
priority
ticks_used
```

are used again.

```
// C
static int mlfq_ticks_since_boost = 0;
```

This keeps track of when the last priority boost happened.

Global priority boost

```
// C
acquire(&tickslock);
```

```

for (struct proc *bp = proc; bp < &proc[NPROC]; bp++) {

```

For every process:

```

    C
    acquire(&bp->lock);
    bp->priority = 0;
    bp->ticks_used = 0;
    release(&bp->lock);

```

So:

```

    priority 0
    |
    | priority 1
    |
    | priority 2
    |
    | priority 3

```

Every 48 ticks, **everyone is moved back to priority 0.**

And their current time-slice counter is reset.

This prevents a process from getting stuck forever at a low priority because higher-priority processes keep arriving.

Check queues in order:

```

q = 0 - highest priority
q = 1
q = 2
q = 3 - lowest priority

```

Then:

```

    C
    if (p->state == RUNNABLE && p->priority == q)

```

Find a process that:

1. is ready to run
2. belongs to the current priority queue

Then:

```

    C
    p->state = RUNNING;

```

and:

```

    C
    c->proc = p;

```

```

| wtime = 0 |
| etime = 0 |
| first_run = -1 |
|
| Scheduler
|
| Process starts running
|
| first_run = ticks
|
| Timer tick
|
| rtime++
| RUNNABLE processes wtime++
| MLFQ ticks_used++
|
| MLFQ time slice ends?
| / \
| YES NO
|
| demote higher priority
| priority process waiting?
| / \
| yield YES NO
|
| yield continue
|
| Process exits
|
| kexit()

```

why do intr_on and then intr_off?

Because the scheduler needs to **allow interrupts while it is getting ready to schedule**, but then **turn them off before doing the actual process-selection/switching work**.

In your code:

```
in C
for (;;) {
    intr_on();
    intr_off();

    // choose a process...
}
```

Why `intr_on()` ?

```
in C
intr_on();
```

This makes interrupts enabled.

The important reason is that if **no process is runnable**, the scheduler eventually does:

```
in C
```

operations.

This prevents a timer interrupt from occurring in the **middle of process selection**.

For example, imagine:

```
scheduler starts
|
| checks P1
|
| checks P2
|
| TIMER INTERRUPT happens here
|
| scheduler state could be changed
|
| continues checking processes
```

That can create race/consistency problems.

So the scheduler wants:

```
enable interrupts
|
| make sure interrupts can wake CPU / avoid deadlock
|
| disable interrupts
|
| safely select a process
|
| switch to it
```

```
#endif
}
}?? some laces say ctr p but it isnt working
```

Yep — if you're inside `xv6` **after running** `make qemu`, `procdump()` is a **kernel function**, so you don't call it like a normal user program.

In stock `xv6`, `procdump()` is normally triggered by **Ctrl-P**.

1. Try Ctrl-P correctly

While the **QEMU xv6 terminal** is focused, press:

Ctrl + P

Don't type `^P`; actually hold `Ctrl` and press `P`.

It should print something like:

```
1 runble init
2 runble sh
3 runble ...
```

With your MLFQ changes:

Switch to another context.

When that process eventually switches back:

```
in C
mycpu()->intena = intena2;
```

restore the **same interrupt state that existed before the switch**.

Simple example

Suppose before `sched()`:

```
interrupts = OFF
mycpu()->intena = 0
```

`sched()` saves:

```
Intena = 0
```

Then switches to the scheduler.

When control eventually comes back:

```
mycpu()->intena = 0
```

So the CPU remembers:

```
#ifdef SCHEDULER_MLFQ
p->ticks_used = 0;
#endif
sched();
}
release(&p->lock);
} do wat
```

Yep — same thing: one-line comments for every line, without changing the code.

```
// C
void
sleep(void)
{
    struct proc *p = myproc();           // get the current process

    acquire(&p->lock);                     // lock the process before changing it

    if (p->chan != 0) {                   // check if the process has a sleep channel
        p->state = SLEEPING;              // mark the process as sleeping
    }
```

```
for (p = proc; p < &proc(NPROC); p++) {
    if (p->state == UNUSED)
        continue;

    if (p->state >= 0 && p->state < NELEM(states) && states[p->state])
        state = states[p->state];
    else
        state = "???";

    //added

#ifdef SCHEDULER_MLFQ
    printf("%d %s %s priority=%d ticks_used=%d\n", p->pid, state, p->name, p->priority, p->ticks_used);

#else
    printf("%d %s %s\n", p->pid, state, p->name);

#endif
}
```

4. Recompile QEMU

After changing kernel code:

```
// Bash
make clean
make qemu
```

Then focus/click inside the QEMU terminal and press:

```
Ctrl-P
```

5. If you're using your Fedora terminal and Ctrl-P still doesn't work

There is an important distinction:

```
Fedora terminal
├─
├─ make qemu
├─
└─ QEMU xv6 console
```

You need to send Ctrl-P to the QEMU/xv6 console, not another terminal/window.

Also, if your shell/terminal is intercepting the key combination, try:

```
Ctrl + P
```

while the QEMU window/console has focus.

Then your flow is

```
make qemu
|
xv6 boots
|
$
|
CLICK INSIDE TERMINAL
|
Ctrl + P
|
consoleintr()
|
procdump()
|
1 rumble init priority:...
2 rumble sh priority:...
...
```

So your `procdump()` code itself isn't necessarily the problem. The screenshot shows that VS Code is catching `Ctrl+P` before `xv6` ever sees it.

If you want the **quickest fix right now**, just run `make qemu` in a separate Fedora terminal and use `Ctrl+P` there.

📄 🔍 ↻ 🔄 ⋮

wait so where exactly do i call my `jobs_refresh_ad_print_activities()`?

Make an `activities.c` file and put this in it:

```
1 C
#include "activities.h"
#include "jobs.h"

void activities(void)
{
    jobs_refresh_and_print_activities();
}
```

Here, you call `jobs_refresh_and_print_activities()` inside `activities()`.

The actual `jobs_refresh_and_print_activities()` function will already be defined in `jobs.c`.

What changed / what needs to be present

The important part is that `main.c` now connects the shell's main loop to `jobs.c`.

1. Include `jobs.h`

```
1 C
#include "jobs.h"
```

This lets `main.c` call the job-control functions. Your uploaded file already has this include. 📄 main

2. Initialize the job system

Inside `main()`:

```
1 C
jobs_init();
}
}
```

prevents the shell from immediately exiting when stopped jobs still exist. 📄 main

5. Notify jobs when the shell exits

There are calls to:

```
1 C
jobs_send_sighup_all();
```

when the shell is actually terminating. 📄 main

One important thing

Do not put the implementation of these functions inside `main.c`:

```
1 C
```

25. Lexer error

```
1 C
if (lex_error)
```

Checks whether lexing failed.

```
1 C
{
    printf("cshell: invalid syntax\n");
    continue;
}
```

Prints an error and goes back to the prompt.

So:

```
input
|
lexer
|
ERROR
|
invalid syntax
|
prompt
```



And this is why xv6's standard scheduler also has the same pattern. Your FIFO and MLFQ schedulers retained it.

📄 🔄 ↺ ⋮

wats my cpu -> intena?

`mycpu()` -> `intena` is basically a saved copy of whether interrupts were enabled before a context switch.

In your xv6 code, `mycpu()` gives you the current CPU's `struct cpu`:

```
/* C
struct cpu *c = mycpu();
```

That `struct cpu` contains:

```
/* C
c->intena
```

which records the CPU's previous interrupt-enabled state.

Where you see it in your scheduler

You have:

1.

30. Cleanup when shell exits

After `break` leaves the loop:

```
/* C
jobs_send_sigup_all();
```

Again sends `SIGUP` to jobs before the shell terminates.

This acts as final job cleanup.

```
/* C
printf("\n");
```

Prints a newline so the terminal ends cleanly.

```
/* C
return 0;
```

Terminates the shell successfully.

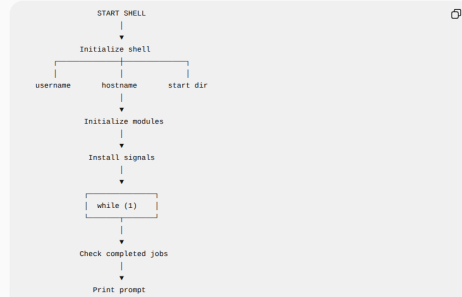
@ conventionally means:

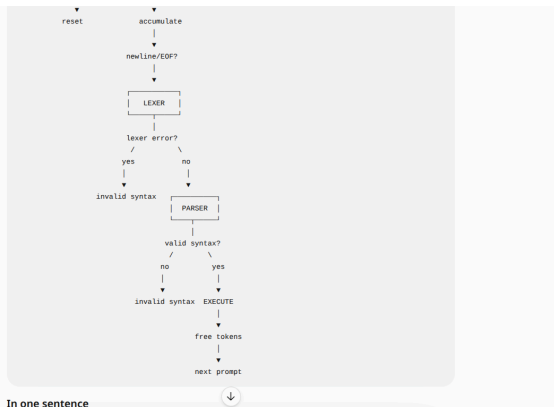
↓

program exited without an error.

Overall flow

The most important thing to understand is the **main loop**:





```
|
v
find "ls"
|
v
execute_pipeline_fg("ls")
|
| WAIT for ls
|
v
ls done
|
v
find "pwd"
|
v
execute_pipeline_fg("pwd")
|
| WAIT for pwd
|
v
pwd done
|
v
find "echo hello"
|
v
execute_pipeline_fg("echo hello")
|
| WAIT
|
v
echo done
```

wait so how do i implement this with my uerrent architecture? liek if i make a sperate spy. c, can u give code for that

To implement the `spy` command in a separate `spy.c` (or integrate it into your custom shell project following standard Linux `proc`-fs parsing conventions), you can design it as a standalone executable or an internal shell command module that inspects `/proc/[pid]`.

Here is how you can implement a complete `spy.c` program matching your requirements:

spy.c

C

```
else if (S_ISDIR(st.st_mode)) strcp(type_str, "DIR");
else if (S_ISREG(st.st_mode)) strcp(type_str, "REG");
else if (S_ISBLK(st.st_mode)) strcp(type_str, "BLK");
else if (S_ISCHR(st.st_mode)) strcp(type_str, "CHR");
else if (S_ISFIFO(st.st_mode)) strcp(type_str, "FIFO");
else if (S_ISLNK(st.st_mode)) strcp(type_str, "LNK");
else if (S_ISSOCK(st.st_mode)) strcp(type_str, "SOCK");
else {
    // If stat fails on symlink target (e.g., deleted files or permissions), default to "REG"
    if (issetlink_path(&st) == 0) {
        if (S_ISLNK(st.st_mode)) strcp(type_str, "REG");
    }
}

// Print format: PID FD TYPE PATH
printf("%-6d %-4s %-6s %s\n", pid, fd_name, type_str, link_path);
}

void inspect_process(pid_t pid) {
    char proc_path[MAX_PATH];
    snprintf(proc_path, sizeof(proc_path), "/proc/%d", pid);

    // Requirement 7: Check if process exists
    struct stat st;
    if (stat(proc_path, &st) != 0) {
        fprintf(stderr, "spy: no such process\n");
    }
}
```

```
FILE *maps_file = fopen(maps_path, "r");
if (!maps_file) {
    char line[512];
    // Simple tracking to print each unique memory-mapped file only once
    char printed_paths[256][MAX_PATH];
    int printed_count = 0;

    while (fgets(line, sizeof(line), maps_file)) {
        char map_file_path[MAX_PATH] = "";
        // Maps format: address perms offset dev inode pathname
        sscanf(line, "%s %s %s %s %s", map_file_path);

        if (map_file_path[0] == '/') { // Absolute path to mapped file
            int already_printed = 0;
            for (int i = 0; i < printed_count; i++) {
                if (strcmp(printed_paths[i], map_file_path) == 0) {
                    already_printed = 1;
                    break;
                }
            }
            if (!already_printed && printed_count < 256) {
                strcpy(printed_paths[printed_count], map_file_path);
                printed_count++;
            }
        }
    }
}
```

```

        target_path[target_len] = '\0';
        print_file_info(pid, entry->d_name, target_path);
    }
    closedir(dir);
}

int main(int argc, char *argv[]) {
    pid_t target_pid;

    // Requirement 8: More than one process ID passed
    if (argc > 2) {
        fprintf(stderr, "spy: invalid syntax\n");
        return 1;
    }

    if (argc == 1) {
        // Requirement 1: No PID provided -> use running shell/current process's pid
        target_pid = getppid();
    } else {

```

```

        if (argc == 1) {
            // Requirement 1: No PID provided -> use running shell/current process's pid
            target_pid = getppid();
        } else {
            // Requirement 2: PID provided
            if (!is_number(argv[1])) {
                fprintf(stderr, "spy: no such process\n");
                return 1;
            }
            target_pid = (pid_t)atoi(argv[1]);
        }

        inspect_process(target_pid);
        return 0;
    }
}

```

Key Implementation Details:

- `readlink()` Utility: Used to resolve symbolic links inside `/proc/[pid]/cwd`, `/proc/[pid]/exe`, and `/proc/[pid]/fd/*` directly into their target filesystem paths.
- **Memory Mappings** (`mem`): Parses `/proc/[pid]/maps` line by line, extracts file paths, and filters out duplicates so each unique mapped file prints only once.
- **Error Handling**: Validates argument lengths (`argc > 2`) for "spy: invalid syntax" and checks process existence (`stat()`) for "spy: no such process".

```

#include <dirent.h>
#include <pwd.h>

#define MAX_PATH 1024

```

- **Includes**: Pulls in standard I/O, string manipulation, POSIX system calls (`readlink`, `getppid`), file status structures (`stat`), directory traversal tools (`opendir`, `readdir`), etc.
- `MAX_PATH`: Defines a safe maximum buffer size (1024 bytes) for handling file paths.

2. Helper: Validating PIDs

```

c
int is_number(const char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] < '0' || str[i] > '9') return 0;
    }
    return 1;
}

```

```

// Helper function to validate if a string is a number
int is_number(const char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] < '0' || str[i] > '9') return 0;
    }
    return 1;
}

```

okay so keeping the spy code and the changes to jobs, can u implement thsi too?

Here is your `snoop.c` implementation along with a detailed explanation of how everything works!

snoop.c Implementation

```

c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "snoop.h"

```

```

void print_file_info(pid_t pid, const char* fd_name, const char* link_path) {
    struct stat st;
    char type_str[16] = "REG";

    if (stat(link_path, &st) == 0) {
        if (S_ISDIR(st.st_mode)) strcpy(type_str, "DIR");
        else if (S_ISCHR(st.st_mode)) strcpy(type_str, "CHR");
        else if (S_ISBLK(st.st_mode)) strcpy(type_str, "BLK");
        else if (S_ISFIFO(st.st_mode)) strcpy(type_str, "FIFO");
        else if (S_ISSOCK(st.st_mode)) strcpy(type_str, "SOCK");
        else if (S_ISREG(st.st_mode)) strcpy(type_str, "REG");
    } else {
        if (lstat(link_path, &st) == 0) {
            if (S_ISLNK(st.st_mode)) strcpy(type_str, "REG");
        }
    }

    printf("%-6d %-6s %-6s %s\n", pid, fd_name, type_str, link_path);
}

```

- o `stat` / `lstat` : Inspects the file metadata at `link_path`.
- o **Macro checks** (`S_ISDIR`, etc.): Determines the file type macro flag (e.g., directory, character device, regular file) and assigns the correct string label ("DIR", "CHR", "REG").
- o `printf` : Outputs the formatted row matching your project specification: PID , FD ,

```

c
void inspect_process(pid_t pid) {
    char proc_path[MAX_PATH];
    snprintf(proc_path, sizeof(proc_path), "/proc/%d", pid);

    struct stat st;
    if (stat(proc_path, &st) != 0) {
        fprintf(stderr, "spy: no such process\n");
        exit(1);
    }
}

```

- o `/proc/[pid]` **check**: Formats the procs directory path for the given PID and runs `stat()` on it. If it fails, it triggers requirement 7 (`spy: no such process`).

```

c
printf("%-6s %-6s %-6s %s\n", "PID", "FD", "TYPE", "PATH");

// 1. Check current working directory (cwd)
char cwd_link[MAX_PATH], cwd_dest[MAX_PATH];
snprintf(cwd_link, sizeof(cwd_link), "/proc/%d/cwd", pid);
ssize_t len = readlink(cwd_link, cwd_dest, sizeof(cwd_dest) - 1);
if (len != -1) {

```

- o **cwd extraction**: Reads the symbolic link at `/proc/[pid]/cwd` using `readlink()` to find the absolute directory path where the process is currently running.

```

c
// 2. Check executable file (txt)
char txt_link[MAX_PATH], txt_dest[MAX_PATH];
snprintf(txt_link, sizeof(txt_link), "/proc/%d/exe", pid);
len = readlink(txt_link, txt_dest, sizeof(txt_dest) - 1);
if (len != -1) {
    txt_dest[len] = '\0';
    print_file_info(pid, "txt", txt_dest);
}

```

- o **txt extraction**: Reads `/proc/[pid]/exe` to locate the binary executable file path that launched the process.

```

        strings(printed_paths[printed_count++], map_file_path, MAX_PATH);
        print_file_info(pid, "mem", map_file_path);
    }

    fclose(maps_file);
}

```

- o **mem extraction:** Opens `/proc/[pid]/maps` which tracks memory regions. It parses out absolute file paths (starting with `/`) and uses a small array tracker (`printed_paths`) to satisfy the rule that each unique memory-mapped file is printed only once.

```
// 4. Check open file descriptors in /proc/pid/fd/
char fd_dir_path[MAX_PATH];
snprintf(fd_dir_path, sizeof(fd_dir_path), "/proc/%d/fd", pid);
DIR *dir = opendir(fd_dir_path);
if (dir) {
    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] == '.') continue;
        char fd_path[MAX_PATH];
        snprintf(fd_path, sizeof(fd_path), "%s/%s", fd_dir_path, entry->d_name);
        fd = open(fd_path, O_RDONLY);
        if (fd == -1) continue;
        fd_set = fd_set_union(fd_set, fd);
        close(fd);
    }
    closedir(dir);
}
```

- File Descriptors (`fd`): Opens the directory `/proc/[pid]/fd`, loops through every file descriptor index (like `0`, `1`, `2`), resolves their symlink targets using `readlink()`, and prints them out.

5. Entry Point (`main`)

```

c
int main(int argc, char *argv[]) {
    pid_t target_pid;

    if (argc > 2) {
        fprintf(stderr, "spy: invalid syntax\n");
        return 1;
    }

    if (argc == 1) {
        target_pid = getppid();
    } else {
        if (!is_number(argv[1])) {
            fprintf(stderr, "spy: no such process\n");
            return 1;
        }
    }
}

```

okay so keeping the spy code and the changes to jobs, can u implement thsi too?

Here is your `snoop.c` implementation along with a detailed explanation of how everything works!

snoop.c Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "snoop.h"
```

```

case 14: return "rt_sigprocmask";
case 15: return "rt_sigreturn";
case 16: return "ioctl";
case 17: return "pread64";
case 18: return "pwrite64";
case 19: return "readv";
case 20: return "writev";
case 21: return "access";
case 22: return "pipe";
case 23: return "select";
case 24: return "sched_yield";
case 25: return "mremap";
case 28: return "madvise";
case 32: return "dup";
case 33: return "dup2";
case 35: return "nanosleep";
case 39: return "getpid";
case 41: return "socket";
case 42: return "connect";
case 43: return "accept";
case 56: return "clone";
case 57: return "fork";
case 58: return "vfork";
case 59: return "execve";
case 60: return "exit";
case 61: return "wait4";
case 62: return "kill";

```

```

        add_syscall_time(current_sys_num, duration);
    }
    in_syscall=0;
}
}
}
}

if(is_attach)
{
    ptrace(PTRACE_DETACH, target_pid, NULL, NULL); //stop tracing the process ad
}

qsort(sys_stats, num_syscalls_tracked, sizeof(sys_stat_t), compare_sys_stats); //

printf("%-11s %-7s %s\n", "syscall", "calls", "time");

for (int i=0; i<num_syscalls_tracked; i++)
{
    printf("%-11s %-7d %.3fs\n", sys_stats[i].name, sys_stats[i].calls, sys_stats[i].time);
}
fflush(stdout);
}

```

Detailed Code Breakdown & Explanation

1. Data Structure & Tracking Architecture (`sys_stat_t`)

- `sys_stat_t` : A custom structure designed to store aggregate metrics for each distinct system call encountered during execution (its name, ID number, total aggregated time, call frequency, and original chronological appearance order).
- `get_syscall_name(long num)` : A static mapping switch-case block that converts raw x86_64 syscall numbers (like 0 for `read`, 1 for `write`, 35 for `nanosleep`) into clean string representations. If a number isn't found, the code gracefully falls back to generating `syscall_N` dynamically (fulfilling Requirement 7).

2. Target Initialization & Process Hooking (`snoop_cmd`)

- **Mode Selection (`-p pid` vs. Command Execution):**
 - If the user specifies `-p <pid>`, it verifies that the process exists using `access()`, and hooks into it using `ptrace(PTRACE_ATTACH, ...)`.
 - If a command is passed instead (e.g., `snoop sleep 1`), it resolves the binary path via the system `PATH` environment variable, forks a child process, calls `ptrace(PTRACE_TRACEME)` to permit parent monitoring, and executes it via `execv()`.

3. Tracing Loop & Timing Execution (`PTRACE_SYSCALL`)

via the `system` `PATH` environment variable, forks a child process, calls `ptrace(PTRACE_TRACEME)` to permit parent monitoring, and executes it via `execv()`.

3. Tracing Loop & Timing Execution (`PTRACE_SYSCALL`)

- o `PTRACE_SETOPTIONS` with `PTRACE_O_TRACESYS6000` : Configures the kernel to tag trap signals originating from system calls cleanly (`SIGTRAP` | `0x80`).
- o **State Toggle (`in_syscall`):**
 - o **Entry (`in_syscall == 0`):** Intercepts the system call entry, reads CPU registers via `ptrace(PTRACE_GETREGS)` to pull the syscall number from `regs.orig_rax`, records a high-resolution starting timestamp via `clock_gettime()`, and switches the state flag to `1`.
 - o **Exit (`in_syscall == 1`):** Intercepts the system call exit, captures the ending timestamp, calculates the exact duration delta, aggregates it via `add_syscall_time()`, and flips the flag back to `0`.

4. Sorting and Reporting

- o `compare_sys_stats` : Implements custom sorting logic for `qsort()` that ranks system calls primarily by **call count descending**, using **first-seen order** as a tie-breaker (fulfilling Requirement 6).
- o **Final Output:** Prints the formatted statistics table displaying the syscall name, total count,



okay so in this i have to implement a sequential execution of this sort : NOW COMMANDS SHLD appear sequentially and The shell must wait for each command to complete before starting the next. How do i implement that in this file?

To implement the sequential execution required in "image_09c0cd.png", the logic relies on two steps:

1. Splitting the command string into distinct blocks every time a semicolon (`OP_SEMI`) is encountered.
2. Executing each block synchronously, meaning the parent shell must block (wait) until the child process finishes before the loop continues to the next command.

```
{
    if (list == NULL || list->count == 0)
    {
        return;
    }

    size_t i = 0;
    while (i < list->count)
    {
        size_t start = i;
```

The function begins by verifying the token list is valid and contains elements. The `while` loop is the main driver that iterates over the entire command line, keeping track of the `start` index for the current command chunk being processed.

```
c
// Advance until a semicolon (sequential) or ampersand (background) is found
while (i < list->count && list->tokens[i].type != OP_SEMI && list->tokens[i]
{
    i++;
```

```
    {
        i++;
    }

    size_t count = i - start;
```

This inner loop is the core of the sequential execution logic. It advances the index `i` until it hits either a semicolon (`OP_SEMI`), an ampersand (`OP_AMP`), or the end of the list. By stopping at semicolons, it isolates `command1` from `command2`, ensuring they are evaluated as completely separate entities.

```
c
int is_bg = 0;

if (i < list->count)
{
    if (list->tokens[i].type == OP_AMP)
    {
        is_bg = 1;
    }
}
```

```

        if (!ok)
        {
            break;
        }
    }
}

```

The command is dispatched. If it is a foreground command (which implies a sequential execution in a ; separated list), `execute_pipeline_fg` is called. The outer `while` loop will halt at this line until `execute_pipeline_fg` completely finishes its internal `waitpid` routine. Once the first command returns, the loop continues and processes the next chunk.

```
execute_pipeline_fg()
```

```

C
int execute_pipeline_fg(const token_list_t *list, const char *full_cmd)
{
    if (!list->count) return 1;
}

```

```

C
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    return 0;
}

```

The shell uses `fork()` to create a duplicate child process. The parent (the shell) will wait, while the child executes the user's program.

```

C
else if (pid == 0) {
    // Child Process

    // Put the child in its own process group
    setpgid(0, 0);
}

```

```

else {
    // Parent Shell Process

    // Assign the child to its own process group to avoid race conditions
    setpgid(pid, pid);

    // Register the job in the shell's tracking system
    pid_t pids[1] = { pid };
    const char *cmd_names[1] = { argv[0] };
    int job_id = jobs_alloc_id();
    jobs_add_pipeline(job_id, pid, pids, cmd_names, 1, full_cmd);

    // Wait synchronously for the child to finish.
    int result = jobs_resume_fg(job_id, 0);

    return result;
}
}

```

In the parent shell (`pid > 0`), `setpgid` is called *again* to prevent a race condition where the shell tries to modify the child's group before the child finishes its own `setpgid` call. The job is added to the shell's internal jobs list. Finally, `jobs_resume_fg` is called. This function contains a `waitpid()` loop that blocks the parent shell from doing anything else until the child finishes,

The shell must print the background job number and process ID in the format: `[job_number] process_id`. This line must be printed before any output produced by the command. Job numbers are assigned session-wide and increase monotonically. They are never reused, even after a job completes. can u implement this in my current code ?

To implement session-wide, monotonically increasing job numbers that are never reused, you need a static global counter for the allocation. To guarantee the `[job_number] process_id` string prints before the background process generates output, the parent process must print and flush `stdout` immediately after forking.

1. Job ID Allocation

```

C
static int next_job_id = 1;

int jobs_alloc_id(void)

```

```

}

{
    return next_job_id++;
}

```

How this works:

- o **Session-Wide Persistence:** The variable `next_job_id` is declared as `static` outside of any function scope. It initializes to `1` when the shell starts and persists in memory for the entire duration of the shell process.
- o **Monotonic Increase & No Reuse:** The postfix increment operator (`++`) returns the current value of `next_job_id` to the caller, and immediately increases the underlying variable by 1. Because there is no logic to decrement or reset this variable, the IDs will strictly increase (1, 2, 3...) and will never be reused, even after a job completes.

2. Background Execution Output

To print the job information before the command produces output, the parent shell process must print and flush `stdout` immediately after calling `fork()`, before it allows the child to fully spin up.

```

c
// Convert tokens to a standard argv array
char *argv[256];
int argc = 0;
for (size_t i = 0; i < list->count; i++) {
    if (list->tokens[i].type == OP_WORD) {
        argv[argc++] = list->tokens[i].text;
    }
}
argv[argc] = NULL;

if (argc == 0) return;

// Block SIGCHLD temporarily to prevent a race condition where the child
// exits before the parent can add it to the jobs list
sigset_t mask, prev_mask;
sigemptyset(&mask);
sigaddset(&mask, SIGCHLD);
sigprocmask(SIG_BLOCK, &mask, &prev_mask);

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    sigprocmask(SIG_SETMASK, &prev_mask, NULL);
    return;
}
else if (pid == 0) {
    // --- Child Process ---

```

```

// 1. Print the required format IMMEDIATELY to ensure it appears before child
printf("[%d] %d\n", job_id, (int)pid);

// 2. Force output to the terminal right now
fflush(stdout);

// 3. Add the job to the internal tracking list
pid_t pids[] = { pid };
const char *cmd_names[] = { argv[0] };
jobs_add_pipeline(job_id, pid, pids, cmd_names, 1, full_cmd);

// 4. Unblock SIGCHLD now that the job is safely registered
sigprocmask(SIG_SETMASK, &prev_mask, NULL);
}
}

```

Key Execution Steps:

- o `printf("[%d] %d\n", job_id, (int)pid);` : Placed in the parent block immediately after `setpgid`.
- o `fflush(stdout);` : Standard output is often line-buffered or block-buffered. Flushing forces the shell to print the job information to the terminal instantly, guaranteeing it appears before the child process executes `execvp` and prints its own output.
- o **Signal Blocking (`sigprocmask`):** When launching background jobs, `SIGCHLD` must be blocked before forking and unblocked after calling `jobs_add_pipeline`. Without this, a

```

c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "ping.h"
#include "jobs.h"
#include <ctype.h>

```

- o Pulls in standard libraries for input/output (`stdio.h`), memory/integer conversions (`stdlib.h`), string manipulation (`string.h`), and character classification (`ctype.h`), along with your custom header files (`ping.h` and `jobs.h`).

2. Input Validation Helper Function

```

c
static int is_non_negative_integer(const char *str)
{
    if (!str || *str == '\0')
    {
        return 0;
    }

```

```
static int is_non_negative_integer(const char *str)
{
    if (!str || *str == '\0') return 0;
    for (size_t i = 0; str[i] != '\0'; i++)
    {
        if (!isdigit((unsigned char)str[i])) return 0;
    }
    return 1;
}

void ping_cmd(const token_list_t *list)
{
    if (!list || list->count != 2)
    {
        printf("ping: invalid syntax\n");
        return;
    }

    const char *target = list->tokens[1]->text;
    const char *sig_str = list->tokens[2]->text;

    // Validate signal number: PIDOT
    if (!is_non_negative_integer(sig_str))
    {
        printf("ping: invalid syntax\n");
        return;
    }
}
```

```

}

int job_id = atoi(target + 1);
if (!jobs_send_signal_to_job(job_id, actual_sig))
{
    printf("ping: no such process found\n");
    return;
}

printf("sent signal %d to %s\n", raw_sig, target);
fflush(stdout);
}
else
{
    if (!is_non_negative_integer(target))
    {
        printf("ping: invalid syntax\n");
        return;
    }

    pid_t pid = (pid_t)atoi(target);
    if (!jobs_send_signal_to_pid(pid, actual_sig))
    {
        printf("ping: no such process found\n");
        return;
    }

    printf("sent signal %d to %s\n", raw_sig, target);
    fflush(stdout);
}
}
```

```

{
    if (!str || *str == '\0')
    {
        return 0;
    }
    for (size_t i = 0; str[i] != '\0'; i++)
    {
        if (!isdigit((unsigned char)str[i]))
        {
            return 0;
        }
    }
    return 1;
}
}
```

- o `is_non_negative_integer`: Checks if a given string is safe, non-empty, and strictly composed of digits (0-9), returning 1 (true) or 0 (false). This is used to validate both signal numbers and PIDs/Job IDs.

3. Command Entry Point (`ping_cmd`) and Syntax Validation

c

The flow should be:

```
main.c
|
| calls
v
jobs.h
|
| declarations
v
jobs.c
|
| implementations
v
job-control functionality
```

And your normal command execution remains:

```
main.c
|
v
lex line()
```

1. Global job storage + job IDs

```
on C
#define MAX_BG_JOBS 256

static job_entry_t jobs_list[MAX_BG_JOBS]; // Stores all background jobs
static int next_job_id = 1; // Next job number
static volatile sig_atomic_t fg_active = 0; // Whether a foreground job is active

int jobs_alloc_id(void)
{
    return next_job_id++; // Return current ID, then increment it
}
```

2. SIGCHLD handler

This runs when a child process changes state, such as exiting, stopping, or continuing.

```
on C
static void sigchld_handler(int sig)
{
    (void)sig; // Signal number is unused
    int saved_errno = errno; // Save errno because handler may change it
    int status;

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (WIFEXITED(status))
        {
            // Child exited
        }
        else if (WIFSIGNALED(status))
        {
            // Child was killed
        }
        else if (WIFSTOPPED(status))
        {
            // Child stopped
        }
    }
}
```

```
#include <ctype.h>
#include "resume.h"
#include "jobs.h"

static int is_all_digits(const char *str)
{
    if (!str || *str == '\0') return 0;
    for (size_t i = 0; str[i] != '\0'; i++)
    {
        if (!isdigit((unsigned char)str[i])) return 0;
    }
    return 1;
}
```

Explanation:

- o **Headers:** Imports standard I/O for printing (`stdio.h`), conversion/allocation (`stdlib.h`), string operations (`string.h`), character checking (`ctype.h`), along with your custom header files (`resume.h` and `jobs.h`).
- o `is_all_digits` **Helper:** This function acts as a safety gate. It verifies that a given string is non-empty and consists strictly of numeric characters (0 through 9). It loops through every character casting it to `unsigned char` (to prevent undefined behavior with `isdigit`) and returns 1 if all characters are digits, or 0 if it encounters any non-digit character.

```

    }
    else
    {
        printf("resume: invalid syntax\n");
        return;
    }

    jobs_resume_fg(job_id, timeout);
}
else
{
    printf("resume: invalid syntax\n");
}
}

```

Explanation:

- o **Foreground Matching:** If the mode string is "fg", the code prepares to bring the suspended job back to the foreground.
- o **Optional Timeout Parsing** (list->count == 5):
 - o Supports an optional timeout extension (e.g., resume %i fg --timeout 5).
 - o If 5 tokens are passed, it checks that tokens[3] explicitly reads --timeout.

```

4
if (!list || list->count < 3)
{
    printf("resume: invalid syntax\n");
    return;
}

const char *target = list->tokens[1].text;
if (!target || target[0] != '%' || !is_all_digits(target + 1))
{
    printf("resume: invalid syntax\n");
    return;
}

int job_id = atoi(target + 1);

const char *mode = list->tokens[2].text;
if (!mode)
{
    printf("resume: invalid syntax\n");
    return;
}
}

```

Explanation:

- o **Token Count Validation:** resume requires at least 3 tokens (e.g., resume %i fg). If

```

    printf("resume: invalid syntax\n");
    return;
}

jobs_resume_fg(job_id, timeout);
}
else
{
    printf("resume: invalid syntax\n");
}
}

```

can u explain ths with one line coments in code block

Yep — here is the same code with simple one-line comments explaining what each line/block does:

```

1  C
2
3  void resume_cmd(const token_list_t *list)
4  {
5      // Check that the token list exists and has at least 3 tokens.
6      if (!list || list->count < 3)
7      {
8          // Print an error if the command does not have enough arguments.
9          printf("resume: invalid syntax\n");
10         return;
11     }
12
13     timeout = 0;
14     }
15     else if (list->count == 5)
16     {
17         if (strcmp(list->tokens[3].text, "--timeout") != 0)
18         {
19             printf("resume: invalid syntax\n");
20             return;
21         }
22         const char *to_str = list->tokens[4].text;
23         if (!is_all_digits(to_str))
24         {
25             printf("resume: invalid syntax\n");
26             return;
27         }
28         timeout = atoi(to_str);
29         if (timeout <= 0)
30         {
31             printf("resume: invalid syntax\n");
32             return;
33         }
34     }
35     else
36     {
37         printf("resume: invalid syntax\n");
38         return;
39     }
40
41     jobs_resume_fg(job_id, timeout);
42 }
43
44
45

```

```

13     timeout = 0;
14     }
15     else if (list->count == 5)
16     {
17         if (strcmp(list->tokens[3].text, "--timeout") != 0)
18         {
19             printf("resume: invalid syntax\n");
20             return;
21         }
22         const char *to_str = list->tokens[4].text;
23         if (!is_all_digits(to_str))
24         {
25             printf("resume: invalid syntax\n");
26             return;
27         }
28         timeout = atoi(to_str);
29         if (timeout <= 0)
30         {
31             printf("resume: invalid syntax\n");
32             return;
33         }
34     }
35     else
36     {
37         printf("resume: invalid syntax\n");
38         return;
39     }
40
41     jobs_resume_fg(job_id, timeout);
42 }
43
44
45

```

/home/yashu/project

might be displayed as:

~/project

```
char cwd[PATH_MAX];
```

Creates a buffer to store the current working directory.

```
if (getcwd(cwd, sizeof(cwd)) == NULL)
```

Gets the current directory.

```
{
    snprintf(out, outsize, "%s");
    return;
}
```

sigemptyset(&sa_int.sa_mask);

Specifies that no additional signals should be blocked while the handler runs.

```
sa_int.sa_flags = 0;
```

No special signal-handling flags.

```
sigaction(SIGINT, &sa_int, NULL);
```

Actually installs the handler for `SIGINT`.

So pressing:

Ctrl-C

causes your handler to run.

10. Installing SIGTSTP handler

```
jobs_list[i].job_id = job_id;
jobs_list[i].pgid = pgid;

jobs_list[i].num_procs =
    num_pids > MAX_PIDS_PER_JOB ? MAX_PIDS_PER_JOB : num_pids;
// Limit number of stored processes

for (int k = 0; k < jobs_list[i].num_procs; k++)
{
    jobs_list[i].procs[k].pid = pids[k]; // Store PID

    strncpy(jobs_list[i].procs[k].cmd_name,
            cmd_names[k],
            sizeof(jobs_list[i].procs[k].cmd_name) - 1);
    // Copy command name safely

    jobs_list[i].procs[k]
        .cmd_name[sizeof(jobs_list[i].procs[k].cmd_name) - 1] = '\0';
    // Guarantee null termination

    jobs_list[i].procs[k].state = PROC_RUNNING; // Initially running
}

if (full_cmd)
{
    strncpy(jobs_list[i].full_cmd, full_cmd,
            sizeof(jobs_list[i].full_cmd) - 1);
    // Save complete command

    jobs_list[i].full_cmd[
        sizeof(jobs_list[i].full_cmd) - 1] = '\0';
}
```

```
break; // Job was successfully added
}
}
```

This function basically creates the shell's record of a background job: job ID → process group → individual PIDs → command names → states. `jobs`

6. Adding a stopped job

```
void jobs_add_stopped(int job_id, pid_t pgid,
                    const pid_t *pids,
                    const char *cmd_names,
                    int num_pids,
                    const char *full_cmd)
{
    jobs_add_pipeline(job_id, pgid, pids, cmd_names, num_pids, full_cmd);
    // First add it normally

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (jobs_list[i].active && jobs_list[i].job_id == job_id)
        {
            jobs_list[i].is_stopped = 1; // Mark whole job stopped
        }
    }
}
```

```
void jobs_refresh_states(void)
{
    int status;

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (!jobs_list[i].active)
            continue; // Ignore inactive jobs

        int any_alive = 0;
        int any_stopped = 0;

        for (int k = 0; k < jobs_list[i].num_procs; k++)
        {
            if (jobs_list[i].procs[k].state == PROC_EXITED)
                continue; // Already finished

            pid_t pid = jobs_list[i].procs[k].pid;

            pid_t res = waitpid(pid, &status,
                                WNOHANG | WUNTRACED);
            // Check current process state

            if (res == pid)
            {
                if (WIFEXITED(status) || WIFSIGNALED(status))
                {
                    jobs_list[i].procs[k].state = PROC_EXITED; // Process ended

                    if (pid == jobs_list[i].pgid)
                    {
                        jobs_list[i].finished = 1;
                        jobs_list[i].exit_status = status;
                    }
                }
            }
        }

        if (any_alive || any_stopped)
        {
            jobs_list[i].state = PROC_RUNNING;
        }
        else
        {
            jobs_list[i].state = PROC_EXITED;
        }
    }
}
```

```

void jobs_check_completed(void)
{
    jobs_refresh_states(); // update states first

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (jobs_list[i].active &&
            jobs_list[i].finished &&
            jobs_list[i].reported &&
            !jobs_list[i].is_stopped)
        {
            const char *first_cmd = jobs_list[i].procs[0].cmd_name;

            printf("Job with pid %d exited %s\n",
                first_cmd,
                (int)jobs_list[i].pgid,
                jobs_list[i].is_signaled ? "abnormally" : "normally");
            // Tell user how job ended

            fflush(stdout);

            jobs_list[i].reported = 1; // Prevent duplicate message
            jobs_list[i].active = 0; // Remove from active jobs
        }
    }
}

```

9. Checking whether stopped jobs exist

```

// C
kill(-job->pgid, signal);

```

means send the signal to the **whole process group**, not just one process. `jobs`

11. Printing active jobs

```

// C
static int compare_job_ptrs(const void *a, const void *b)
{
    const job_entry_t *j1 = *(const job_entry_t *)a;
    const job_entry_t *j2 = *(const job_entry_t *)b;

    return j1->job_id - j2->job_id; // Sort by job number
}

```

```

// C
void jobs_refresh_and_print_active(void)
{
    jobs_refresh_states(); // update everything first

    job_entry_t *active_jobs[MAX_BG_JOBS];
    int count = 0;

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (jobs_list[i].active)
        {
            active_jobs[count++] = &jobs_list[i];
        }
    }
}

```

```

}

return NULL; // Job not found
}

```

13. Checking whether a PID is tracked

```

// C
int jobs_pid_is_tracked(pid_t pid)
{
    jobs_refresh_states(); // update process states

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (jobs_list[i].active)
        {
            for (int k = 0; k < jobs_list[i].num_procs; k++)
            {
                if (jobs_list[i].procs[k].pid == pid &&
                    jobs_list[i].procs[k].state != PROC_EXITED)
                {
                    return 1; // PID belongs to an active job
                }
            }
        }
    }

    return 0; // PID not tracked
}

```

So:

15. Resume a job in foreground

This is the more complicated resume .

```

int jobs_send_signal_to_pid(pid_t pid, int sig)
{
    if (!jobs_pid_is_tracked(pid))
    {
        return 0; // PID isn't tracked
    }

    kill(pid, sig); // Signal only this process

    return 1; // Success
}

```

So the difference is:

```
jobs_send_signal_to_job()
```

```
// Execute without waiting
} else
{
    jobs_set_fg_active(1); // Mark foreground execution

    int ok = execute_pipeline_fg(jobs_tokens,
                                  full_cmd_str);

    // Execute and wait

    jobs_set_fg_active(0); // Foreground execution finished

    jobs_check_completed(); // Report completed background jobs
}

if (!ok)
{
    break; // Stop after foreground failure
}
}
```

This gives you the overall flow:

```

      command line
      |
      v
split at ; or &
      |
+-----+-----+
|                                     |
| ' & '                               |
|                                     |
+-----+-----+

```

22. Handling Ctrl-D / incomplete interactive input

Checks whether:

- accumulated input doesn't end in newline
- Input comes from an interactive terminal

This handles the special behavior you've implemented for `ctrl-d`.

Keep the partially entered text and wait for more input.

23. Remove newline

40 C

instead of the full path.

This checks whether `home_dir` is a parent directory of the current directory.

For example:

The first part checks that `cwd` starts with `home_dir`.

The second part:

ensures that you're actually at a subdirectory rather than something like:

[/home/yashu2](#)



can u suggest yt resources for me to learn about the topics needed in this. Can u give me a list of what to implement

Here is a comprehensive checklist of what you need to implement for Part D (Sequential and Background Execution), along with recommended YouTube resources to help you master the core concepts.

Part 1: What to Implement

D1: Sequential Execution (`command1 ; command2 ; ... ; commandN`)

1. **Command Parser/Tokenizer:** Split the input line by the semicolon (`;`) delimiter into individual commands.
2. **Sequential Loop:** Iterate through the list of commands and execute them one by one.
3. **Blocking Execution (`wait`):** Ensure the parent shell waits for each child process to fully complete using `wait()` or `waitpid()` before launching the next command in the sequence.

To understand the system calls (`fork` , `exec` , `waitpid`) signal handling, and asynchronous reaping required for this assignment, check out these helpful YouTube videos:

- o For understanding `fork()` and process creation fundamentals:
 - o The `fork()` function in C by CodeVault
 - o `fork()` and `exec()` System Calls by Neso Academy
- o For handling signals, `SIGCHLD`, and preventing zombie processes (`waitpid` with `WNOHANG`):
 - o Short introduction to signals in C by CodeVault
 - o Search channels like CodeVault for advanced systems programming topics on handling `SIGCHLD` and asynchronous process reaping.



Linux system programming fork exec waitpid signal handling c



Fork() system call tutorial
hotdayr
Video tutorial illustrating the `fork()` system call. In this video, we illustrate the basics of `fork()`. We demonstrate how `fork()` creates ...

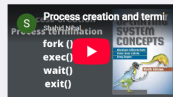
`fork()` and `exec()` System Calls



The `fork()` function in C
CodeVault
Check out our Discord server:
<https://discord.gg/NFxtBNY>.



Short introduction to signals
CodeVault
Check out our Discord server:
<https://discord.gg/NFxtBNY>.



Process creation and termination -
`fork()`, `exec()`, `wait()` and `exit()`
Shahid Rehal
Process Creation in UNIX: `fork()`, `exec()`, `wait()`, and Process Management Explained In this video, we explore how process ...

```
void cpu_bound_work(void) { // func for heavy cpu work
for (volatile int i=0; i<200000000; i++) { // big ol loop to waste time
// Heavy CPU computation
}
```

It is a trick to force the processor to actually burn time for the benchmark test.

If you wrote a normal empty loop like `for (int i = 0; i < 200000000; i++) {}`, a modern compiler (like GCC) would look at it, realize the loop accomplishes absolutely nothing, and delete the entire thing during the optimization phase. Your "heavy" process would finish instantly.

Adding the `volatile` keyword tells the compiler: "Do not optimize this variable. You must perform every read and write exactly as written."

This forces the CPU to physically count to 200,000,000 step-by-step. It eats up real CPU cycles (ticks), successfully faking a heavy computational workload so you can test how your OS scheduler handles a process that refuses to share the processor.

🔄 🔍 📄 ...

wait so wat do i change in my proc.c to add response time

To track response time accurately, you need to modify three specific functions inside `kernel/proc.c`.

1. `allocproc()`: Initialize the tracking variables

When a new process is created, you must set its initial creation time and reset the `first_run` flag so it doesn't inherit garbage data from a dead process.

```
c
// Inside allocproc()
found:
p->pid = allocpid();
p->state = USED;

// Add these lines right here:
acquire(&tickslock);
```

```
}

p->state = RUNNING;
c->proc = p;
// ... existing context switch code ...
```

3. `waitx()`: Calculate and return the math to user space

Update the function signature to take the 4th argument (`retime`), calculate the response time (`first_run - ctime`), and safely copy it back to the user space pointer.

```
c
// Update signature to include the 4th argument
int waitx(uint64 addr, uint64 rtime, uint64 wtime, uint64 retime)
{
// ... loop through processes ...
if (pp->state == ZOMBIE) {
pid = pp->pid;

// Add the retime copyout block here with the others:
int resp = (pp->first_run >= 0) ? (pp->first_run - pp->ctime) : 0;
if (copyout(p->pagetable, p->sz, retime, (char *)&resp, sizeof(resp)) <
release(&swait_lock);
return -1;
}
}

// ... existing copyout blocks for xstate, rtime, wtime ...
pp->parent = 0;
r->proc(pp);
```

(`first_run - ctime`), and safely copy it back to the user space pointer.

```
c
// Update signature to include the 4th argument
int waitx(uint64 addr, uint64 rtime, uint64 wtime, uint64 retime)
{
// ... loop through processes ...
if (pp->state == ZOMBIE) {
pid = pp->pid;

// Add the retime copyout block here with the others:
if (retime != 0) {
int resp = (pp->first_run >= 0) ? (pp->first_run - pp->ctime) : 0;
if (copyout(p->pagetable, p->sz, retime, (char *)&resp, sizeof(resp)) <
release(&swait_lock);
return -1;
}
}

// ... existing copyout blocks for xstate, rtime, wtime ...
pp->parent = 0;
r->proc(pp);
```

```
acquire(&tickslock);
p->ctime = ticks;
release(&tickslock);
p->first_run = -1; // -1 means it has not been scheduled yet
```

2. `scheduler()`: Record the first time it hits the CPU

Whenever the scheduler picks a process to run, check if it's the very first time. If it is, log the current time.

```
c
// Inside scheduler(), right when u find a RUNNABLE process:
if (p->state == RUNNABLE) {
// Add this block before changing state to RUNNING:
if (p->first_run == -1) {
acquire(&tickslock);
p->first_run = ticks;
release(&tickslock);
}

p->state = RUNNING;
c->proc = p;
```

okay i have no idea how xv6 works. can u suggest yt videos to explainw at exctly it is and how to implement smething in that architecture

What is xv6?

xv6 is a modern re-implementation of Version 6 UNIX (v6) created by MIT for teaching operating system principles.

- o It is written in ANSI C and runs on multiprocessor RISC-V (or x86) architectures.
- o It models a clean, simple, traditional Unix-like kernel containing essential OS concepts: processes, page tables, system calls, file systems, trap handling, and device drivers—all in just a few thousand lines of readable source code.
- o Usually, you run and test xv6 inside an emulator like QEMU.

Recommended YouTube Resources to Understand xv6

To get up to speed on what xv6 is, how it boots, and how its code architecture is structured, watch these videos:

- o For understanding the big picture of booting & the codebase architecture:
 - o **W2_L1: Booting: introduction to QEMU & xv6 by IIT Madras - B.S. Degree Programme** — Great introductory walkthrough of how xv6 launches inside QEMU.

- o For learning how processes and underlying structures work in xv6:
 - o **CS 134 OS—9: processes in xv6 by Neil Rhodes** — Breaks down how the process control block (`struct proc`), process states, and scheduling work internally.
 - o **CS 134 OS—6: xv6 First Process by Neil Rhodes** — Explains how the kernel spins up its very first user-space process (`init`).

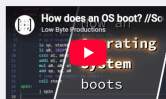
How to Implement Something in xv6

When you are tasked with adding a feature or system call into xv6, you generally follow a strict pattern across the kernel and user space. For instance, if you were adding a feature or system call, you would typically touch these core files:

1. **User Space Interface** (`user/user.h` & `user/usys.pl`):
 - o Declare your user-facing function signature in `user.h`.
 - o Add a stub in `usys.pl` so that the user program traps safely into the kernel.
2. **System Call Numbers** (`kernel/syscall.h`):
 - o Assign a unique integer identifier (macro) for your new system call.
3. **Kernel Dispatch Table** (`kernel/syscall.c`):
 - o Map the syscall number to the actual underlying C function handler in the kernel.
4. **Kernel Implementation** (`kernel/sysproc.c` or `kernel/sysfile.c`):

- o Create a new C file under the `user/` directory (e.g., `user/mytest.c`), call your function, and add it to the `UPROGS` list in the `Makefile` so it gets compiled into the xv6 disk image when you run `make qemu`.

YouTube
Xv6 operating system explained what is it tutorial MIT operating system course



How does an OS boot? //Source Dive// 001
Low Byte Productions
In this installment of //Source Dive//, we're learning about the **xv6 Operating System**. Specifically the low-level boot code that gets ...



Operating Systems Course for Beginners
freeCodeCamp.org
Learn fundamental and advanced operating system concepts in 25 hours. This course will give you a comprehensive ...



W2_L1: Booting: introduction to

YouTube

MIT xv6 operating system lectures code walkthrough

CS 134 OS—6: xv6 First Process

Neil Rhodes

Day 6 of Harvey Mudd College Operating Systems class.

Bootstrapping xv6 OS on RISC-V (Arty A7 Demo)

vrarmstrong

This video showcases a project focused on creating a cost-effective, open-source educational framework by successfully booting ...

CS 134 OS—15: Crash Recovery and

Continuing in xv6

can u explain wat everything in thsi makefile does : K=kernel
U=user

```
OBJ$ = \
    $K/entry.o \
    $K/start.o \
    $K/console.o \
    $K/print.o \
    $K/uart.o \
    $K/kalloc.o \
    $K/spinlock.o \
    $K/string.o \
    $K/main.o \
    $K/vm.o \
    $K/proc.o \
    $K/swtch.o \
    $K/trampoline.o \
    $K/trap.o \
    $K/syscall.o \
    $K/sysproc.o \
    $K/bio.o \
    $K/fs.o \
    $K/log.o \
    $K/sleeplock.o \
    ...
```

```
\
    exit 1; \
fi

.PHONY: fmt
fmt:
    clang-format -i $(wildcard kernel/*.[ch] user/*.[ch] mkfs*.c)
```

Yep. Think of this Makefile as the recipe that turns your xv6 source code into:

```
C/Assembly source files
+
+ compiler
+
+ .o files
+
+ linker
+
+ kernel/kernel
+
+ disk image fs.img
+
+ qemu
+
+ running xv6
```

I'll go through it section by section, especially the parts relevant to your scheduler project.

```
Makefile
kernel
user
```

These are simply variables.

Instead of repeatedly writing:

```
kernel/proc.o
user/printf.o
```

you can write:

```
Makefile
$K/proc.o
$U/printf.o
```

So:

```
Makefile
$K/proc.o
```

becomes:

```
kernel/proc.o
user/printf.o
```

becomes:

```
user/printf.o
```

2. OBJ\$

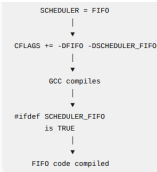
```
Makefile
OBJ$ = \
    $K/entry.o \
    $K/start.o \
    $K/console.o \
    ...
    $K/proc.o \
    ...
    $K/virtio_disk.o
```

This is the list of object files that make up the kernel.

For example:

```
kernel/proc.o
```

gets compiled into:



This is **compile-time selection**.

It does NOT mean you change schedulers while xv6 is running.

19. Disable PIE

You have:

```
🔗 Makefile
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[-f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
```



fs.img is effectively xv6's virtual disk.

33. Dependency inclusion

```
🔗 Makefile
#include kernel/*.d user/*.d
```

Remember:

```
🔗 Makefile
CFLAGS += -MD
```

generated .d dependency files.

This line tells Make to include them.

The `+` before `include` means:

Don't complain if the files don't exist yet.

This says:

If the user hasn't specified `CPUS`, use 3 CPUs.

Therefore:

```
🔗 Bash
make qemu
```

uses:

```
3 virtual CPUs
```

You can override it:

```
🔗 Bash
make qemu CPUS=1
```

or:

```
🔗 Bash
make qemu CPUS=4
```

This is a trick to refer to your scheduler without having scheduler behavior depend on the

how do i modify the above Makefile to support a SCHEDULER macro that selects the scheduling algorithm to compile in?

You need to add a small conditional block to your Makefile so that `SCHEDULER` can be passed from the command line and converted into C preprocessor macros.

1. Add this to your Makefile

You already have:

```
Makefile
CFLAGS += -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2
CFLAGS += $(DETFLAGS)
CFLAGS += -march=rv64gc
...
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-prot
```

Immediately after this section, add:

```
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-prot
```

Immediately after this section, add:

```
Makefile
ifdef SCHEDULER
CFLAGS += -DS$(SCHEDULER) -DSCHEDULER_$(SCHEDULER)
endif
```

So that part of your Makefile becomes:

```
Makefile
CFLAGS += -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-prot

ifdef SCHEDULER
CFLAGS += -DS$(SCHEDULER) -DSCHEDULER_$(SCHEDULER)
endif

# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
```

Suppose you run:

```
Bash
make qemu SCHEDULER=FIFO
```

Make substitutes `FIFO` for `$(SCHEDULER)`:

```
Makefile
CFLAGS += -DFIFO -DSCHEDULER_FIFO
```

These are passed to the C compiler.

Therefore, your C code can select the scheduler with:

```
C
#ifdef SCHEDULER_FIFO
// FIFO scheduler implementation
#endif
```

Similarly, if you run:

```
Bash
make qemu SCHEDULER=MLFQ
```

The `SCHEDULER_FIFO` form is generally cleaner for scheduler-selection code because it makes the purpose of the macro explicit:

```
C
#ifdef SCHEDULER_FIFO
// ...
#elif defined(SCHEDULER_RR)
// ...
#elif defined(SCHEDULER_MLFQ)
// ...
#endif
```

The plain `FIFO` macro is also available if your existing code expects it.

4. How you use it

First clean the old compiled objects:

```
Bash
make clean
```

Then choose your scheduler:

```
Bash
make qemu SCHEDULER=FIFO
```

Or:

```
Bash
make clean
make qemu SCHEDULER=MLFQ
```

or whatever scheduler names your implementation defines.

Important: use:

```
Bash
SCHEDULER=FIFO
```

with no spaces around `=`.

The key idea is:

```
make qemu SCHEDULER=FIFO
├── Makefile
├── CFLAGS += -DFIFO -DSCHEDULER_FIFO
├── C compiler
└── #ifdef SCHEDULER_FIFO
```

so can u make a file that creates several test processes that consume the CPU for diff amounts of time before giving it up (e.g. simulate different CPU-burst lengths).

Yes. For the **MLFQ analysis**, the idea is to create several child processes with different CPU burst lengths. Some processes should run for a long time, while others finish their CPU work quickly. This lets you observe how the MLFQ scheduler treats processes that repeatedly consume CPU versus those with shorter bursts.

For example:

- **PID/process 0:** long CPU burst.
- **PID/process 1:** sleeps/yields repeatedly (I/O-bound)
- **PID/process 2:** long CPU burst.
- **PID/process 3:** sleeps/yields repeatedly

The following `schedulertest.c` creates exactly those kinds of processes and then uses `waitx()` to measure their runtime and waiting time.

```
C
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define NUM_PROCESSES 4
```

```

// C
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define NPROC 4

void cpu_bound_work(void) {
    for (volatile int i = 0; i < 200000000; i++) {
        // heavy CPU computation
    }
}

void io_bound_work(void) {
    for (int i = 0; i < 20; i++) {
        pause(1); // voluntary yield / sleep
    }
}

int main(int argc, char *argv[]) {
    printf("=== Scheduler Benchmark Test (NPROC=%d) ===\n", NPROC);

    for (int i = 0; i < NPROC; i++) {
        int pid = fork();
        if (pid == 0) {
            if (i % 2 == 0) {
                cpu_bound_work();
            } else {
                io_bound_work();
            }
            exit(0);
        }
    }
}

```

```

// C
int total_wtime = 0;
int total_ttime = 0;

for (int i = 0; i < NPROC; i++) {
    int r, w;
    int pid = wait(&r, &w);
    int turnaround = r + w;

    total_rtime += r;
    total_wtime += w;
    total_ttime += turnaround;

    printf("Process PID %d: Runtime = %d ticks, WaitTime = %d ticks, TurnaroundTime = %d ticks\n",
        pid, r, w, turnaround);
}

printf("=== Summary Statistics ===\n");
printf("Average Response / Running Time: %d ticks\n", total_rtime / NPROC);
printf("Average Waiting Time: %d ticks\n", total_wtime / NPROC);
printf("Average Turnaround Time: %d ticks\n", total_ttime / NPROC);

exit(0);
}

```

What this tests

`cpu_bound_work()` keeps the process on the CPU for a large number of iterations, producing a long CPU burst.

`io_bound_work()` calls `pause(1)` repeatedly, so the process voluntarily gives up the CPU and sleeps. This represents an I/O-bound process.

Before running the test, you should compile the program with the following command:

what's procdump? how does it work? how do I implement this?

`procdump()` is basically **xv6's built-in process debugger**. It prints the current process table so you can see what every process is doing.

In your MLFQ project, it is especially useful because you need to verify things like:

- Which queue/priority is each process in?
- How many ticks has it used in its current time slice?
- Is a process **RUNNABLE**, **RUNNING**, **SLEEPING**, etc.?
- Are processes getting moved to lower-priority queues?
- Are they being boosted back to the highest queue?

1. How `procdump()` works

In your `kernel/proc.c`, you already have:

```

// C
void
procdump(void)
{
    // ...
    [RUNNABLE] = "runable",
    [RUNNING] = "run",
    [DORMIE] = "zombie"
};

struct proc *p;
char *state;

printf("%d\n", 0);
for (p = proc; p < &proc[NPROC]; p++) {

```

`proc` is the global process table:

```

// C
struct proc proc[NPROC];

```

So this loop:

```

// C
for (p = proc; p < &proc[NPROC]; p++)

```

goes through every possible process slot.

Then:

```

// C
if (p->state == UNUSED)
    continue;

```

RUNNING
SLEEPING

2. What you need to add for MLFQ

Your assignment specifically wants the debugging output to include:

| PID, name, state, current priority/queue number; and bookkeeping data such as ticks consumed.

You already have the two MLFQ fields in your `struct proc` because your code uses:

```

// C
p->priority
p->ticks_used

```

You also initialize them in `allocproc()`:

```

// C
p->priority = 0;
p->ticks_used = 0;

```

So the simplest implementation is to make `procdump()` print those fields **only when MLFQ is being compiled**.

Add this at the end of the process loop:

```

// C

```

3. So the complete `procdump()` should look like this

```
1 // C
2
3 void
4 procdump(void)
5 {
6     static char "states[]" = {
7         // clang-format off
8         [UNUSED] = "unused",
9         [USED] = "used",
10        [SLEEPING] = "sleep ",
11        [RUNNABLE] = "runable",
12        [RUNNING] = "run ",
13        [ZOMBIE] = "zombie"
14        // clang-format on
15    };
16
17    struct proc *p;
18    char "state";
19
20    printf("\n");
21    for (p = proc; p < &proc[NPROC]; p++) {
22        if (p->state == UNUSED)
23            continue;
24
25        if (p->state == 0 && p->state < NELEM(states) && states[p->state])
26            state = states[p->state];
27        else
28            state = "???";
29
30        #ifdef SCHEDULER_MLQ
```

4. What the output means

When you compile with:

```
1 SCHEDULER_MLQ
```

you might press `Ctrl+P` and get something like:

```
1 init runble priority=0 ticks_used=0
2 sh run priority=0 ticks_used=1
3 test runble priority=2 ticks_used=4
4 test sleep priority=1 ticks_used=0
```

You can interpret that as:

Process	State	Priority	ticks_used
PID 1	RUNNABLE	0	0
PID 2	RUNNING	0	1
PID 3	RUNNABLE	2	4
PID 4	SLEEPING	1	0

```
1 // C
2
3 #ifdef SCHEDULER_MLQ
```

means:

"Only compile the following code if `SCHEDULER_MLQ` is defined."

So when you build your MLFQ version:

```
1 SCHEDULER_MLQ
```

you get:

```
1 PID name state priority ticks_used
```

But for normal `xv6`/Round-Robin, you don't try to print MLFQ-specific information.

That's important because `priority` and `ticks_used` are specifically bookkeeping fields you've added for your MLFQ implementation.

6. How `Ctrl+P` actually helps you test MLFQ

The workflow is basically:

```
1 Start xv6
2
3 Run your scheduler test
4
```

```

    |
    | Press Ctrl+P
    |
    | procdump()
    |
    | Walk through proc[]
    |
    | Print each process's state + MLFQ information
    |

```

You can repeatedly press **Ctrl+P** while your test processes are running and watch their state change.

For example, you might see:

```
test runble priority=0 ticks_used=4
```

then later:

```
test runble priority=1 ticks_used=2
```

That would give you evidence that the CPU-bound process is being **demoted from queue 0 to queue 1** after consuming its allotted time.

And after a priority boost, you should eventually see:

```
test runble priority=0 ticks_used=0
```

which shows that the process was **boosted back to the highest-priority queue** and its slice counter **reset**.

So `procdump()` itself **doesn't implement the MLFQ**. It is a **window into your scheduler's internal**



how do i implement these in the existing xv6 repo

To implement this Multi-Level Feedback Queue (MLFQ) specification into your existing xv6 repository, you will need to map out the rules shown in your assignment images using the files we discussed earlier (`proc.h`, `proc.c`, and `trap.c`).

Here is how you implement these specific requirements step-by-step:

Step 1: Update Process Control Block Fields (`proc.h`)

You need to ensure your process structure (`struct proc`) contains the necessary variables to track the queue level and CPU ticks consumed. Inside `struct proc` in `proc.h`, make sure you have:

Step 2: Initialize New Processes into Queue 0 (`proc.c`)

According to rule 21.31 (New process placement), any newly allocated process must start at the highest priority queue (0) with its tick count zeroed out. Inside `allocproc()`:

```

c
p->priority = 0;
p->ticks_used = 0;

```

Step 3: Handle Time Slices, Preemption, and Aging (`trap.c`)

Rule 21.2 defines the exact time slices for each priority queue: Queue 0 = 1 tick, Queue 1 = 4 ticks, Queue 2 = 8 ticks, and Queue 3 = 16 ticks.

Inside `usertrap()`, when a timer interrupt occurs (`which_dev == 2`), you must handle time-slice exhaustion (Rule 4), preemption by higher-priority processes (Rule 2), and priority boosting/aging (Rule 7).

Replace or adapt your timer-interrupt section in `usertrap()` to look like this:

```

c
if (which_dev == 2) {
    static const int slice_ticks[] = {1, 4, 8, 16};
    p->ticks_used++;

    // Rule 4: Time-slice exhaustion
    if (p->ticks_used >= slice_ticks[p->priority]) {
        p->ticks_used = 0;
        if (p->priority < 3) {
            p->priority++; // Move down to next lower queue
        }
        yield();
    } else {
        // Rule 2: Strict priority selection / preemption check
        int should_yield = 0;
        for (int q = 0; q < p->priority; q++) {
            for (struct proc *hp = proc; hp < &proc[NPROC]; hp++) {
                if (hp->state == RUNNABLE && hp->priority == q) {

```

Replace or adapt your timer-interrupt section in `usertrap()` to look like this:

```

c
if (which_dev == 2) {
    static const int slice_ticks[] = {1, 4, 8, 16};
    p->ticks_used++;

    // Rule 4: Time-slice exhaustion
    if (p->ticks_used >= slice_ticks[p->priority]) {
        p->ticks_used = 0;
        if (p->priority < 3) {
            p->priority++; // Move down to next lower queue
        }
        yield();
    } else {
        // Rule 2: Strict priority selection / preemption check
        int should_yield = 0;
        for (int q = 0; q < p->priority; q++) {
            for (struct proc *hp = proc; hp < &proc[NPROC]; hp++) {
                if (hp->state == RUNNABLE && hp->priority == q) {

```

The shell must print the background job number and process ID in the format: [job_number] process_id. This line must be printed before any output produced by the command. Job numbers are assigned session-wide and increase monotonically. They are never reused, even after a job completes. can u implement this in my current code ?

To implement session-wide, monotonically increasing job numbers that are never reused, you need a static global counter for the allocation. To guarantee the [job_number] process_id string prints before the background process generates output, the parent process must print and flush stdout immediately after forking.

1. Job ID Allocation

```
static int next_job_id = 1;
```

the highest priority queue (0) with its tick count zeroed out. inside allocproc():

```
p->priority = 0;
p->ticks_used = 0;
```

Step 3: Handle Time Slices, Preemption, and Aging (trap.c)

Rule 2.12 defines the exact time slices for each priority queue: Queue 0 = 1 tick, Queue 1 = 4 ticks, Queue 2 = 8 ticks, and Queue 3 = 16 ticks.

Inside usertrap(), when a timer interrupt occurs (which_dev == 2), you must handle time-slice exhaustion (Rule 4), preemption by higher-priority processes (Rule 2), and priority boosting/aging (Rule 7).

Replace or adapt your timer-interrupt section in usertrap() to look like this:

```
for (struct proc *p = proc; p < &proc[NPROC]; p++) {
    if (p->state == RUNNABLE && p->priority == 0) {
        should_yield = 1;
        break;
    }
    if (should_yield) break;
}
if (should_yield) {
    yield();
}
```

Step 4: Implement the MLFQ Scheduler Rules (proc.c)

The scheduler must enforce strict priority selection (always running from the highest non-empty queue) and priority boosting every 48 ticks (anti-starvation).

Modify your scheduler() function under the #ifdef defined(SCHEDULER_MLFQ) block:

```
switch(&c->context, &p->context);
mycpu()->intena = 0;
c->proc = 0;
found = 1;
release(&p->lock);
if (found) break;
if (found) break;
if (!found) {
    spin_waitidle("wfl");
}
```

Step 5: Preserve Priority on Voluntary Yields (proc.c)

According to rule 2.13.5 (Voluntary yield), if a process blocks for I/O or yields before its time slice expires, its priority must remain unchanged when it becomes runnable again. When a

process goes to sleep or voluntarily yields, ensure you do not reset its priority level, though resetting ticks_used upon sleeping is standard practice:

According to rule 2.13.5 (Voluntary yield), if a process blocks for I/O or yields before its time slice expires, its priority must remain unchanged when it becomes runnable again. When a process goes to sleep or yields, ensure you do not reset its priority level, though resetting ticks_used upon sleeping is standard practice:

```
void
sleep(void)
{
    struct proc *p = myproc();

    acquire(&p->lock);
    if (p->chan != 0) {
        p->state = SLEEPING;
        #ifdef SCHEDULER_MLFQ
        p->ticks_used = 0; // Reset ticks used, but DO NOT change p->priority
        #endif
        sched();
    }
    release(&p->lock);
}
```

```
for (struct proc *p = proc; p < &proc[NPROC]; p++) {
    if (p != p) {
        acquire(&p->lock);
        if (p->state == RUNNABLE) {
            p->priority = 0;
            release(&p->lock);
        }
    }
}

#ifdef SCHEDULER_FIFO
// Non-preemptive FIFO: do not yield on timer interrupt
#else
static const int slice_ticks[] = {1, 4, 8, 16};
p->ticks_used++;

// Check if time slice for current queue has expired
if (p->ticks_used >= slice_ticks[p->priority]) {
    p->ticks_used = 0;
    if (p->priority < 3) {
        p->priority++;
    }
    yield();
} else {
    // Check if a higher-priority process is RUNNABLE
    int should_yield = 0;
}
```

```
static int mlfq_ticks_since_boost = 0;

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for (;;) {
        intr_on();
        intr_off();

        // Rule 7: Priority boosting (anti-starvation) every 48 ticks
        acquire(&ticks_lock);
        if (ticks == mlfq_ticks_since_boost >= 48) {
            mlfq_ticks_since_boost = ticks;
            for (struct proc *p = proc; p < &proc[NPROC]; p++) {
                acquire(&p->lock);
                p->priority = 0;
                p->ticks_used = 0;
                release(&p->lock);
            }
        }
    }
}
```

```
#ifdef SCHEDULER_MLFQ
    printf("pid %d priority %d ticks_used %d\n", p->pid, p->name, state, p->priority);
#else
    printf("pid %d %s\n", p->pid, state, p->name);
#endif
```

Explanation: Modifies the output of procdump (triggered by typing ^P on the console) to print out the process priority and ticks used if compiled with the MLFQ flag. C

4. System Call Plumbing (sysproc_2.c, syscall_2.h, syscall_2.c, user_2.h, usys_2.pl)

To fully support the new waitx system call, multiple files were updated to plumb the call from user space down to the kernel.

sysproc_2.c: Wraps the system arguments before passing them to the kernel-side implementation. C

```
C
```

how do i implement this as a function to my existing code.
DO NOT TELL ME TO INTEGRATE IT TO MY CODE THATS
LEADING TO A LOT OF ERRORS PLS JST MAKE A FUNCTION
PLS

To satisfy the requirements outlined in "image_0a2647.png", the logic is divided into two specific functions: a signal handler to catch the terminations asynchronously, and a checker function to handle the required printing.

Here is how the functions address the specific rules:

- Requirements 6 & 7: The `sigchld_handler` function catches the `SIGCHLD` signal when a background process state changes. It uses `waitpid(..., WNOHANG, ...)` to reap the child process immediately without blocking the main shell process. It then saves whether the process exited normally (`WIFEXITED`) or was killed by a signal (`WIFSIGNALED`).
- Requirements 8 & 9: The `jobs_check_completed` function evaluates the flags set by the handler. If the job finished, it checks the `is_signaled` flag. A ternary operator seamlessly swaps between the words "abnormally" and "normally" to match the exact string format `<command_name> with pid <pid> exited [normally/abnormally]`. It then marks `<command_name>` with `pid <pid> exited [normally/abnormally]`. It then marks the job as reported so it is only printed once.

```
c
static void sigchld_handler(int sig)
{
    (void)sig;
    int saved_errno = errno;
    int status;

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        if (jobs_list[i].active && !jobs_list[i].finished)
        {
            for (int k = 0; k < jobs_list[i].num_procs; k++)
            {
                pid_t pid = jobs_list[i].procs[k].pid;

                // Requirement 7: waitpid with WNOHANG guarantees non-blocking reaps
                pid_t res = waitpid(pid, &status, WNOHANG | WUNTRACED | WCONTINUED);

                if (res == pid)
                {
                    // Requirement 8 & 9: Determine if it exited normally or if it
```

```
C
}

void jobs_check_completed(void)
{
    jobs_refresh_states();

    for (int i = 0; i < MAX_BG_JOBS; i++)
    {
        // Find jobs that have finished but haven't been printed to the user yet
        if (jobs_list[i].active && jobs_list[i].finished && !jobs_list[i].reported &&
            const char *first_cmd = jobs_list[i].procs[0].cmd_name;

        // Requirements 8 & 9: Print exact format based on how the process terminated
        printf("%s with pid %d exited %s\n",
            first_cmd,
            (int)jobs_list[i].pgid,
            jobs_list[i].is_signaled ? "abnormally" : "normally");

        fflush(stdout);

        // Mark as reported to prevent duplicate printing
        jobs_list[i].reported = 1;
        jobs_list[i].active = 0;
    }
}
```

```
{
uint64 p, rtime, wtime, rettime;
argadd(0, &p);
argadd(1, &rtime);
argadd(2, &wtime);
argadd(3, &rettime);
return wait(p, rtime, wtime, rettime);
}
```

`syscall_2.h` & `syscall_2.c`: Defines the unique integer mapping for `waitx` and adds it to the system call table. H=1

```
c
// in syscall_2.h
#define SYS_waitx 23 //added

// in syscall_2.c
extern uint64 sys_waitx(void); //added
[SYS_waitx] = sys_waitx, //added
```

```
// in syscall_2.c
extern uint64 sys_waitx(void); //added
[SYS_waitx] = sys_waitx, //added
```

`usys_2.pl` & `user_2.h`: Automatically generates the assembly stubs for the `ecall` interface and exposes the function signature to user programs. PL=1

```
Perl
// in usys_2.pl
entry("waitx"); #added
```

```
c
// in user_2.h
```

```
printf("resume: invalid syntax\n");
return;
}

jobs_resume_fg(job_id, timeout);
}
else
{
    printf("resume: invalid syntax\n");
}
}
```

can u explain this with one line comments in code block

Yep — here is the same code with simple one-line comments explaining what each line/block does:

```
us C
void resume_cmd(const token_list_t *list)
{
    // Check that the token list exists and has at least 3 tokens.
    if (!list || list->count < 3)
    {
        // Print an error if the command does not have enough arguments.
        ...
    }
}
```

```
}

if (strcmp(mode, "bg") == 0)
{
    if (list->count != 3)
    {
        printf("resume: invalid syntax\n");
        return;
    }
    jobs_resume_bg(job_id);
}
else if (strcmp(mode, "fg") == 0)
{
    int timeout = 0;
    if (list->count == 3)
    {
        timeout = 0;
    }
    else if (list->count == 5)
    {
        if (strcmp(list->tokens[3].text, "--timeout") != 0)
        {
            printf("resume: invalid syntax\n");
        }
    }
}
```

2. Process Control Block Additions (`proc.h`)

The original PCB additions for benchmarking and MLFQ scheduling. `H`

```
c
// MLFQ fields
int priority;           // Current MLFQ priority queue (0-3)
int ticks_used;         // Ticks consumed in current priority time slice

// Benchmarking metrics
int ctime;              // Creation time (ticks when created)
int rtime;              // Total running time (ticks spent RUNNING)
int wtime;              // Total waiting time (ticks spent RUNNABLE)
int etime;              // End/Exit time (ticks when exited)
int first_run;          // Time when first scheduled to run (-1 if not yet run)
};
```

```
static const int slice_ticks[] = {1, 4, 8, 16};
p->ticks_used++;

// Check if time-slice for current queue has expired
if (p->ticks_used >= slice_ticks[p->priority]) {
    p->ticks_used = 0;
    if (p->priority < 3) {
        p->priority++;
    }
    yield();
} else {
    // Check if a higher-priority process is RUNNABLE
    int should_yield = 0;
    for (int q = 0; q < p->priority; q++) {
        for (struct proc *hp = proc; hp < &proc[NPROC]; hp++) {
            if (hp->state == RUNNABLE && hp->priority == q) {
                should_yield = 1;
                break;
            }
        }
        if (should_yield) break;
    }
    if (should_yield) {
        yield();
    }
}
```

```
if (hp->state == RUNNABLE && hp->priority == 0) {
    should_yield = 1;
    break;
}
}
if (should_yield) break;
}
if (should_yield) {
    yield();
}
}
#else
yield();
#endif
}
```

Explanation:

- o This block intercepts timer interrupts (`which_dev == 2`) inside `usertrap()`. `C`
- o It increments the active running time (`rtime`) for the current process and loops through the process table to increment the waiting time (`wtime`) for all other `RUNNABLE` processes. `C`
- o Depending on compilation flags, it modifies scheduling behavior: under FIFO, it prevents yielding; under MLFQ, it tracks ticks against specific queue time slices (`1`, `4`, `8`, or `16`).

```

C
// MLFQ fields
int priority;           // Current MLFQ priority queue (0-3)
int ticks_used;         // Ticks consumed in current priority time slice

// Benchmarking metrics
int ctime;              // Creation time (ticks when created)
int rtime;              // Total running time (ticks spent RUNNING)
int wtime;              // Total waiting time (ticks spent UNRUNNABLE)
int etime;              // End/Exit time (ticks when exited)
int first_run;          // Time when first scheduled to run (-1 if not yet run)
};

```

Explanation:

- o Adds vital tracking fields to the Process Control Block (struct proc).
- o priority and ticks_used manage multi-level feedback queue states.
- o ctime, rtime, wtime, etime, and first_run store timing metrics used to evaluate process performance and calculate turnaround/response times via the waitx system call.

```

C
// Process Initialization (allocproc)
p->priority = 0;
p->ticks_used = 0;

acquire(&tickslock);
p->ctime = ticks;
release(&tickslock);
p->rtime = 0;
p->wtime = 0;
p->etime = 0;
p->first_run = -1;

// Exit Logging (kexit)
acquire(&p->lock);
acquire(&tickslock);
p->etime = ticks;
release(&tickslock);
p->xstate = status;
p->state = ZOMBIE;
release(&wait_lock);

// Extended wait system call (waitx)
int

```

```

C
// Schedulers (FIFO & MLFQ) and sleep adjustments

```

Explanation:

- o **Initialization (allocproc):** Sets up a new process into the highest priority queue (0), records its birth tick (ctime), and resets performance timers.
- o **Exit Tracking (kexit):** Captures the final system ticks (etime) right before a process transitions into a ZOMBIE state.
- o **waitx System Call:** Extends traditional child-waiting functionality by copying out collected tracking metrics (rtime , wtime , exit status) back to user-space pointers using copyout.
- o **Schedulers & Sleep:** Implements alternative scheduling policies (FIFO selecting by oldest ctime, MLFQ with 48-tick global boosting) and ensures voluntary sleeps clear time-slice usage.

```
uint64
sys_waitx(void)
{
    uint64 p, rtime, wtime;
    argaddr(0, &p);
    argaddr(1, &rtime);
    argaddr(2, &wttime);
    return waitx(p, rtime, wtime);
}
```

Explanation:

- o Acts as the kernel-side system call interface for `waitx`. C
- o It fetches the user-space pointers for the exit status variable, run-time tracker, and wait-time tracker from the hardware registers via `argaddr` before invoking the kernel function `waitx()`. C

```
jobs_set_fg_active();
// Tell the job-control system that a foreground
// command is currently running.

int ok = execute_pipeline_fg(
    &cmd_tokens,
    full_cmd_str
);
// Execute the current command in the foreground.
//
// IMPORTANT:
// execute_pipeline_fg() does not immediately return
// while the command is still running.
//
// It waits for the foreground process/pipeline to
// finish (or stop).
//
// Therefore, the next iteration of this while loop
// does NOT happen until this command has completed.

jobs_set_fg_active(0);
// The foreground command has now finished/stopped,
// so tell the job system that there is no longer
// an active foreground command.

jobs_check_completed();
// Check whether any background jobs completed while
// the foreground command was running.
```

```
C
extern uint64 sys_waitx(void);

static uint64 (*syscalls[])(void) = {
    // ...
    [SYS_waitx] = sys_waitx,
};
```

Explanation:

- o Maps the system call number macro (`SYS_waitx`) to its corresponding kernel implementation function pointer (`sys_waitx`) inside the global dispatch table. C

7. user.h

Reference Code Snippet:

```
C
p->proc = curproc;
release(&wait_lock);

// Extended wait system call (waitx)
int
waitx(uint64 addr, uint64 rtime, uint64 wtime)
{
    // ... core loop handling zombie cleanup and metric copyout ...
}

// Schedulers (FIFO & MLFQ) and sleep adjustments
```

Explanation:

- o Initialization (`allocproc`): Sets up a new process into the highest priority queue (`0`), records its birth tick (`ctime`), and resets performance timers. C
- o Exit Tracking (`kexit`): Captures the final system ticks (`etime`) right before a process transitions into a `ZOMBIE` state. C
- o `waitx` System Call: Extends traditional child-waiting functionality by copying out collected tracking metrics (`rtime`, `wttime`, exit status) back to user-space pointers using `copyout`. C
- o Schedulers & Sleep: Implements alternative scheduling policies (FIFO selecting by oldest `ctime`, MLFQ with 48-tick global boosting) and ensures voluntary sleeps clear time-slice usage. C

4. sysproc.c